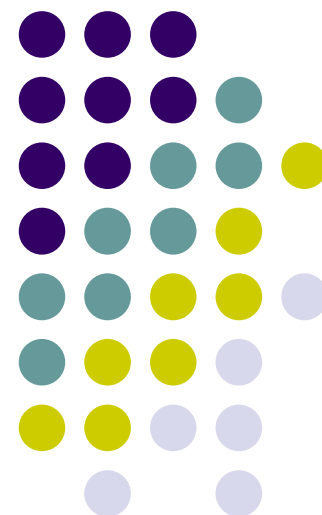
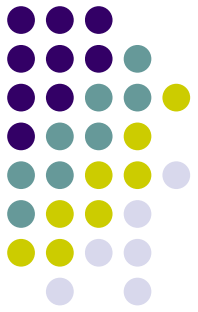


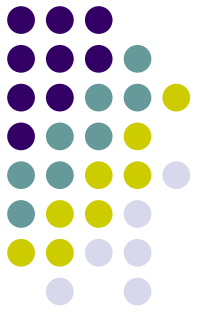
# Ch5 进程通信





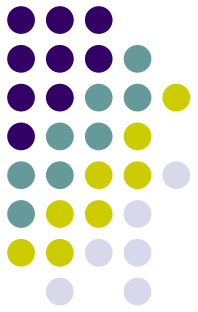
# 本章内容

- 信号
- 管道
- **IPC**通信（消息队列、共享内存、信号量）
- 网络通信



# Part 1 信号

- 信号概念
- 信号相关命令
- signal 函数
- 信号发送
  - kill, raise
- alarm和 pause 函数

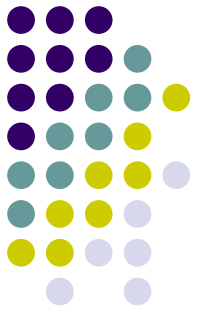


# 1.1 信号概念

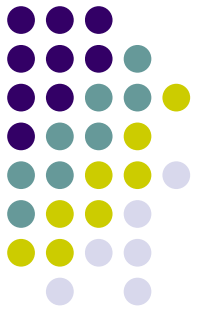
- 信号 外部异步事件
  - Software interrupt 软件中断
  - Mechanism for handling asynchronous events 异步事件
  - Having a name (beginning with SIG)
  - Defined as a positive integer (in <signal.h>)
- 信号产生
  - 按终端键，硬件异常，kill(2)函数，kill(1)命令，软件条件，...

# kill -l

查看信号类型

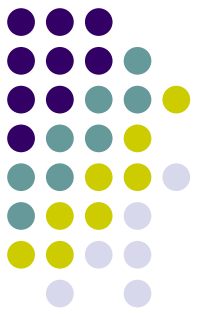


- **SIG**信号（1-31）是从**UNIX**系统中继承下来的称为不可靠信号(也称为非实时信号)。
- **SIGRTMIN~SIGRTMAX**是为了解决前面“不可靠信号”问题而进行更改和扩充的信号，称为可靠信号(也称为实时信号)。
- 可靠信号(实时信号)：支持排队，发送用户进程一次就注册一次，发现相同信号已经在进程中注册，也要再注册。
- 不可靠信号(非实时信号)：不支持排队，发送用户进程判断后注册，发现相同信号已经在进程中注册，就不再注册，忽略该信号。



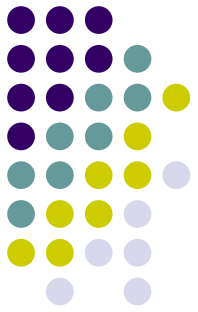
# Signals in Linux/UNIX—kill -l

| 名称             | 说明                            | 默认操作      |
|----------------|-------------------------------|-----------|
| SIGABRT        | 进程异常终止（调用abort函数产生此信号）        |           |
| <b>SIGALRM</b> | <b>超时（alarm）</b>              | <b>终止</b> |
| SIGFPE         | 算术运算异常（除以0，浮点溢出等）             |           |
| <b>SIGHUP</b>  | <b>连接断开</b>                   | <b>终止</b> |
| SIGILL         | 非法硬件指令                        |           |
| <b>SIGINT</b>  | <b>终端终端符(Clt-C)</b>           | <b>终止</b> |
| <b>SIGKILL</b> | <b>立即结束程序运行（不能被捕捉、阻塞或忽略）</b>  | <b>终止</b> |
| <b>SIGPIPE</b> | <b>向没有读进程的管道写数据</b>           |           |
| <b>SIGQUIT</b> | <b>终端退出符(Clt-^)</b>           | <b>终止</b> |
| <b>SIGTERM</b> | <b>终止（由kill命令发出的系统默认终止信号）</b> | <b>退出</b> |
| <b>SIGUSR1</b> | <b>用户定义信号</b>                 | <b>退出</b> |
| <b>SIGUSR2</b> | <b>用户定义信号</b>                 | <b>退出</b> |



# Signals in Linux/UNIX (cont'd)

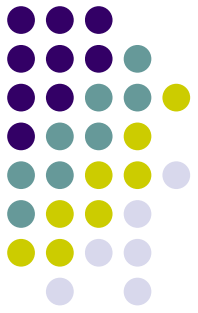
| 名称             | 说明                    | 默认操作  |
|----------------|-----------------------|-------|
| SIGSEGV        | 无效存储访问（段违例）           |       |
| <b>SIGCHLD</b> | <del>子进程停止或退出</del>   | 忽略    |
| <b>SIGCONT</b> | 使暂停进程继续               | 继续/忽略 |
| <b>SIGSTOP</b> | 暂停一个进程（不能被捕捉、阻塞或忽略）   | 终止    |
| <b>SIGTSTP</b> | 终端挂起符( <b>Clt-Z</b> ) | 停止进程  |
| SIGTTIN        | 后台进程请求从控制终端读          |       |
| SIGTTOU        | 后台进程请求向控制终端写          |       |



# 信号处理

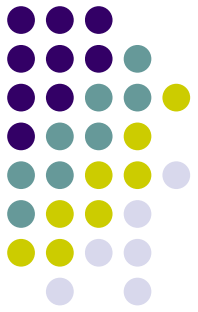
- 忽略信号
  - 不能忽略的信号：
    - SIGKILL, SIGSTOP
    - 一些硬件异常信号
- 执行系统默认动作
- 捕捉信号





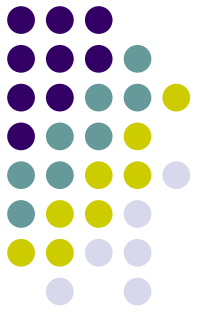
## 1.2 信号相关命令

- Kill
  - 暂停 `kill -STOP <pid>`
  - 恢复 `kill -CONT <pid>`
  - 终止 `kill -KILL <pid>`



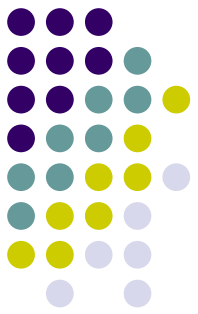
## 1.3 信号相关函数

| 函数           | 功能                                            |
|--------------|-----------------------------------------------|
| kill         | 发送信号给 <u>进程或进程组</u>                           |
| raise        | 发送信号给 <u>进程自身</u>                             |
| alarm        | 定时器时间到，向进程发送 <b>SIGALRM</b> 信号                |
| <u>pause</u> | <u>没有捕捉信号前一直将进程挂起</u>                         |
| signal       | 捕捉信号 <b>SIGINT</b> , <b>SIGQUIT</b> 时执行信号处理函数 |



## 1.3 信号相关函数

| 函数                              | 功能                    |
|---------------------------------|-----------------------|
| <code>sigemptyset</code>        | 初始化信号集合为空             |
| <code>sigfillset</code>         | 初始化信号集合为所有信号集合        |
| <code>sigaddset</code>          | 将 <u>指定信号</u> 加入到指定集合 |
| <u><code>sigdelset</code></u>   | 将 <u>指定信号</u> 从信号集中删除 |
| <u><code>sigprocmask</code></u> | 判断 <u>检测或更改信号屏蔽字</u>  |



# 1 信号发送——kill & raise

- kill: send signal to a process

```
#include <sys/types.h>
```

```
#include <signal.h>
```

```
int kill(pid_t pid, int sig);
```

(Returned Value: 0 if success, -1 if failure)

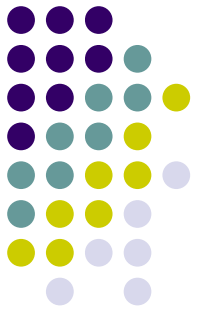
The “pid” parameter

pid > 0: 发送信号给进程id为pid的进程。

pid = 0: 发送信号给与自己同组的所有进程。

pid = -1: 发送系统内所有进程

pid < -1: 发送给进程组号为-pid的所有进程。



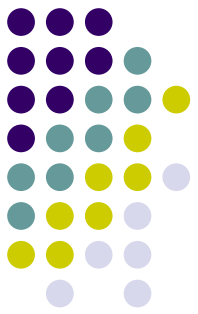
# 信号发送——kill & raise

- raise: send a signal to the current process

`#include <signal.h>`

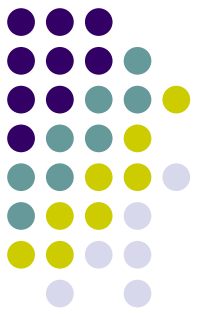
`int raise(int sig);`

(Returned Value: 0 if success, -1 if failure)



## 例5-1

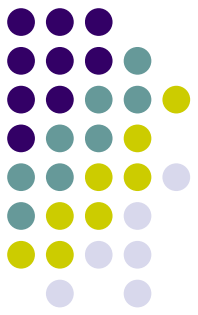
- 设计一程序，要求用户进程复制出一个子进程，父进程向子进程发出**SIGKILL**信号，子进程收到此信号，结束子进程的运行。
- 分析：用户进程**fork**子进程后，子进程使用**raise**函数发送**SIGSTOP**信号，使自己暂停；父进程使用**kill**函数向子进程发送**SIGKILL**信号，子进程收到信号结束。



```
int main ()
{ pid_t result; int ret; int newret;
  result=fork();
  if(result<0)
  { perror("创建子进程失败");
    exit(1);  }
  else if (result==0)
  { raise(SIGSTOP); /*调用
    exit(0);  }
  else
  { printf("子进程的进程号(PID)是: %d\n",result);
    if((waitpid(result,NULL,WNOHANG))==0)
    { if((ret=kill(result,SIGKILL))==
      printf("用kill函数返回值是: %d\n",ret);
      进程号: %d\n",ret,result);
      else { perror("kill函数结束子进程");
        }  }
  }
}
```

**Waitpid (pid, NULL, WNOHANG) 没有子进程终止立即返回, 返回值为0**

由此例可知, 系统调用**kill**函数和**raise**函数, 都是简单地向某一进程发送信号。**kill**函数用于给特定的进程或进程组发送信号, **raise**函数用于向一个进程自身发送信号。



## 2 信号处理——“signal” 函数

- Installs a new signal handler for the signal with number `signum`.

```
#include <signal.h>
```

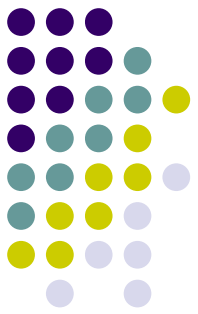
```
typedef void (*sighandler_t)(int);
```

```
sighandler_t signal(int signum, sighandler_t handler);
```

(Returned Value: the previous handler if success,  
`SIG_ERR` if error)

- The “`handler`” parameter
  - a user specified function, or
  - `SIG_DFL`, or
  - `SIG_IGN`

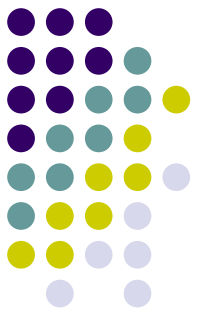




# The "signal" function (cont'd)

- Program example

```
static void sig_usr(int);  
int main(void)  
{  
    if (signal(SIGUSR1, sig_usr) == SIG_ERR)  
        err_sys("can't catch SIGUSR1");  
    if (signal(SIGUSR2, sig_usr) == SIG_ERR)  
        err_sys("can't catch SIGUSR2");  
  
    for ( ; ; )  
        pause();  
}
```



### 3 alarm & pause 函数

- alarm: <sup>启动定时</sup> set an alarm clock for delivery of a signal

`#include <unistd.h>`

`unsigned int alarm(unsigned int seconds);`

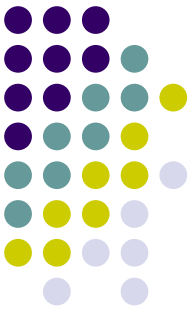
(Returned value: 0, or the <sup>或</sup> number of seconds remaining of previous alarm)

- <sup>等待信号</sup> pause: wait for a signal

`#include <unistd.h>`

`int pause(void);`

(Returned value: -1, errno is set to be EINTR)



# alarm & pause 函数

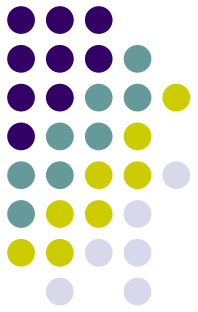
- Program example

- 实现 “sleep” 函数:

```
signal(SIGALRM,fun)
```

```
alarm()
```

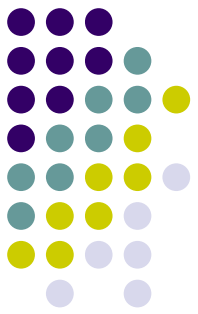
```
pause()
```



## 4. 信号阻塞

有时既不希望进程在接收到信号时立刻中断进程的执行，也不希望此信号完全被忽略掉，而是延迟一段时间再去调用信号处理函数，这个时候就需要信号阻塞来完成。

屏蔽：接收到，但不处理



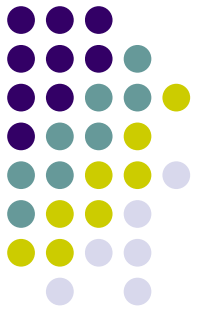
# 信号集处理函数

- `#include <signal.h>`
- `int sigemptyset(sigset_t *set);`
- `int sigfillset(sigset_t *set);`
- `int sigaddset(sigset_t *set,int signum);`
- `int sigdelset(sigset_t *set,int signum);`
- `int sigismember(const sigset_t *set,int signum);`

参数: **set** 信号集 ; **signum** 信号

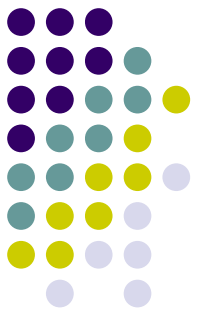
返回值: 若成功返回0, 若出错返回-1。

**sigismember**若真返回1, 若假返回0, 若出错返回-1。



## 例5-2 signalset1

```
#include <stdio.h>
#include <signal.h>
#include <stdlib.h>
main()
{   sigset_t *set;
    set=(sigset_t*)malloc(sizeof(sigset_t)); 分配内存
    sigemptyset(set);
    sigaddset(set, SIGUSR1); 加入信号
    sigaddset(set, SIGINT); 信号已经存在，不需要重复加入
    if((sigismember(set, SIGUSR1))!=1) printf("SIGUSR1\n");
    if((sigismember(set, SIGUSR2))!=1) printf("SIGUSR2\n");
    if((sigismember(set, SIGINT))!=1) printf("SIGINT\n");
}
```



# 信号集处理函数 **sigprocmask**

- `#include <signal.h>`
- 功能：检测或更改信号屏蔽字
- 函数：`int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);`

- 参数：`how` 操作方式 `set`

`how` 决定函数的操作方式。

`SIG_BLOCK`: 增加一个信号集合到当前进程的阻塞集合之中。

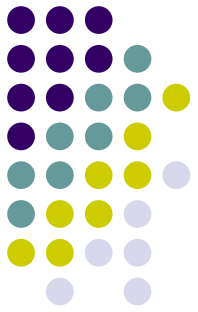
`SIG_UNBLOCK`: 从当前的阻塞集合之中删除一个信号集合。

`SIG_SETMASK`: 将当前的信号集合设置为信号阻塞集合。

- 返回值：若成功返回0，若出错返回-1。

执行失败

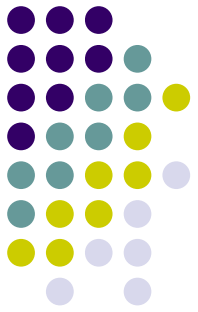




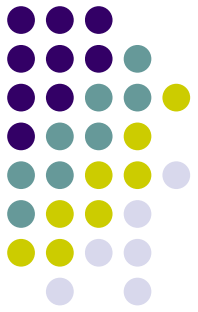
## 例5-3 signalset2

```
#include <stdio.h>
#include <signal.h>
#include <stdlib.h>
main()
{
    sigset_t *set;
    set=(sigset_t*)malloc(sizeof(sigset_t));
    sigemptyset(set);
    sigaddset(set,SIGINT);
    sigprocmask(SIG_SETMASK,set,NULL);
    //也可以使用: sigprocmask(SIG_BLOCK,set,NULL);
    while(1);
}
```





- 程序先定义信号集**set**，然后把信号**SIGINT**添加到**set**信号集中，最后把**set**设置为信号阻塞集合。运行程序时，进程进入死循环。按“**ctrl+c**”系统并没有中断程序，**SIGINT**信号屏蔽掉了。按“**ctrl+z**”来结束程序。



# 信号实例

```
void handler()  
{    alarm(1);    }  
main(int argc,char *argv[])  
{    char buf[BUFSIZE];  
    signal(SIGALRM,handler);  
    alarm(1);
```

*直接 open 子进程*

```
    sfd = open(argv[1],O_RDONLY);
```

```
    while(1)
```

```
    {    pause();
```

```
        len = read(sfd,buf,10);
```

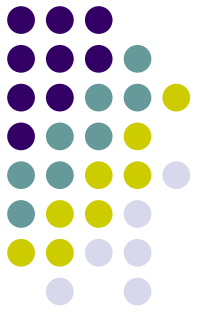
```
        ret = write(1,buf,len);
```

```
    }  
}
```

*出错错误*

```
while((len = read(sfd,buf,BUFSIZE))<0)  
{    if (errno == EINTR)  
        continue; 被信号打断,  
    perror("read()"); 重新打开  
    break;  
}  
if (len==0) break;
```

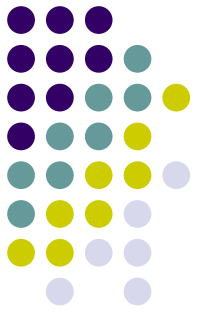
```
while(len>0)  
{    ret = write(1,buf,len);  
    if (ret < 0)  
    {    if (errno == EINTR)  
            continue;  
        perror("write");  
        exit(1);  
    }  
    len -= ret;  
}
```



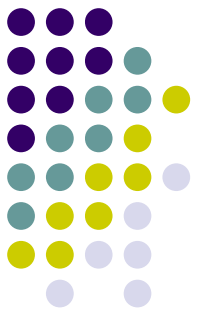
# 掌握的知识点

- 信号的概念
- 信号处理方式
- 常见的信号及处理方式
- kill, signal, raise, pause, alarm
- 信号集处理函数sigprocmask

# Part 2 pipe管道



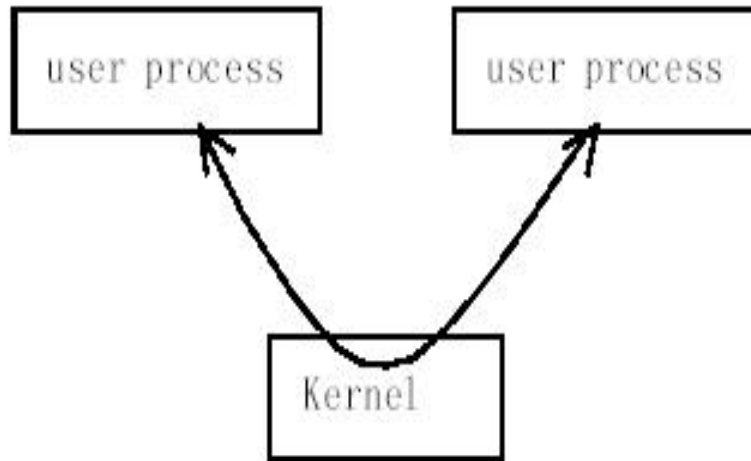
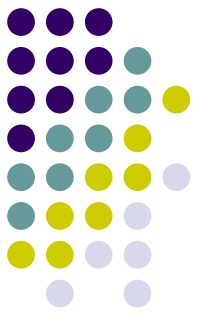
- IPC 概念
- pipe
- popen pclose
- FIFO



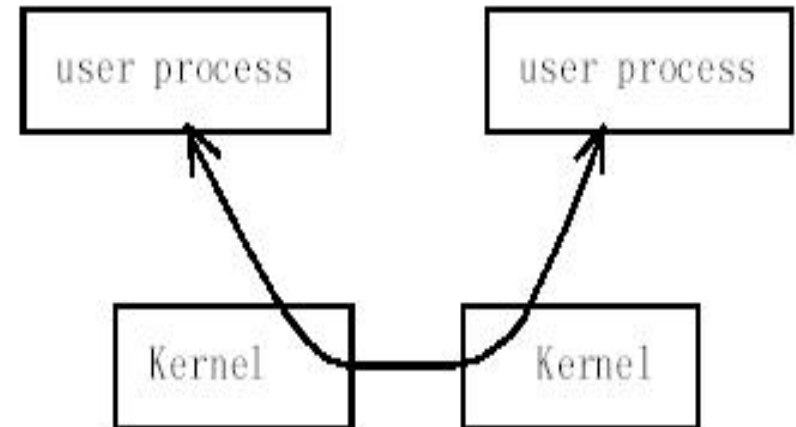
## 2.1 进程间通信

- IPC: Inter-Process Communication
- IPC mechanisms
  - shared file
  - signal
  - pipe, FIFO (named pipe), message queue, semaphore, shared memory
  - socket

# IPC illustrations

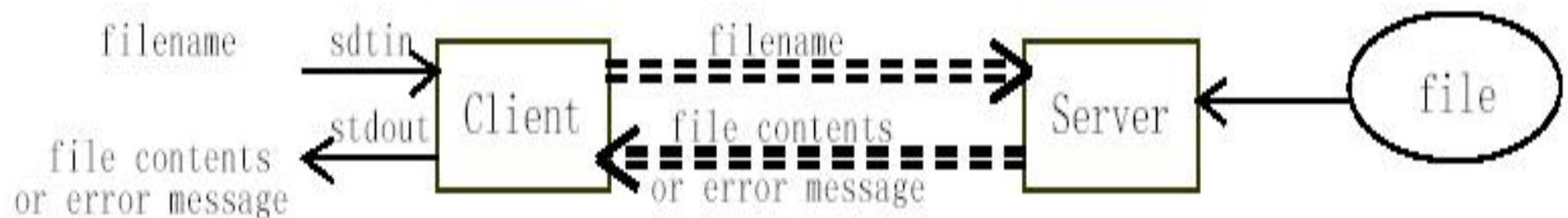
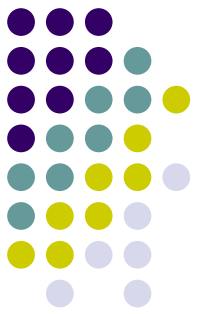


IPC on one host



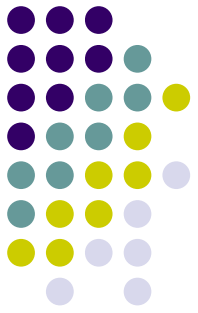
IPC on different hosts:  
Network Programming

# Simple Client-Server or IPC model



Client reads filename from stdin,  
write it to the IPC channel, read the  
file contents from the IPC channel

Server reads filename from the  
IPC channel, opens file, write  
the contents on the IPC channel



## 2.2 pipe 概念

- Pipe

无名管道

半双工

必须有共同祖先

- Pipe mechanism in a shell

- e.g. cmd1 | cmd2

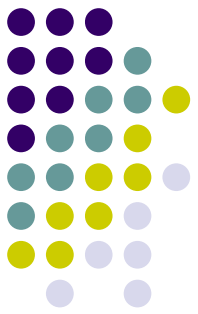
- Pipe is half-duplex

- 管道只能在共同祖先的进程间使用

- 管道也是文件

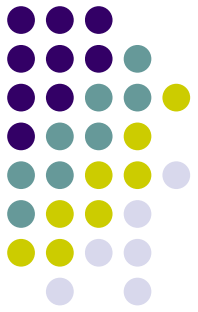
- 命名管道(FIFO)



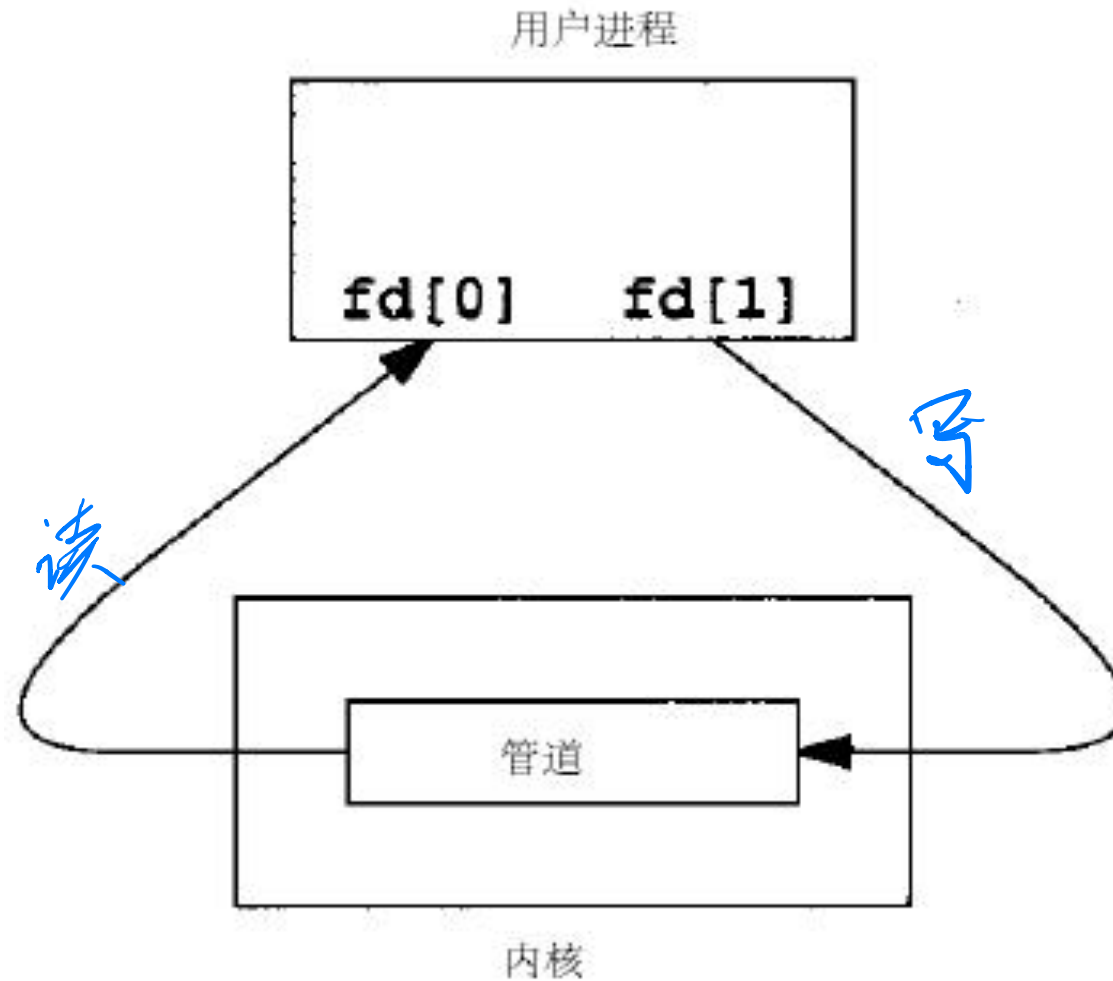


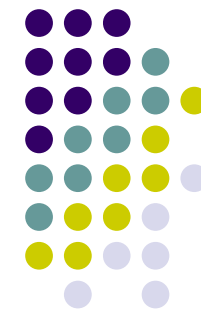
# pipe 函数

- The pipe function: create a pipe  
`#include <unistd.h>`  
`int pipe(int filedes[2]);`  
(Returned value: 0 if success, -1 if failure)
- A pipe: First In, First Out
  - filedes[0]:read, filedes[1]: write



# 单个进程使用管道





# 父子进程使用管道

先开管道  
后开程序。

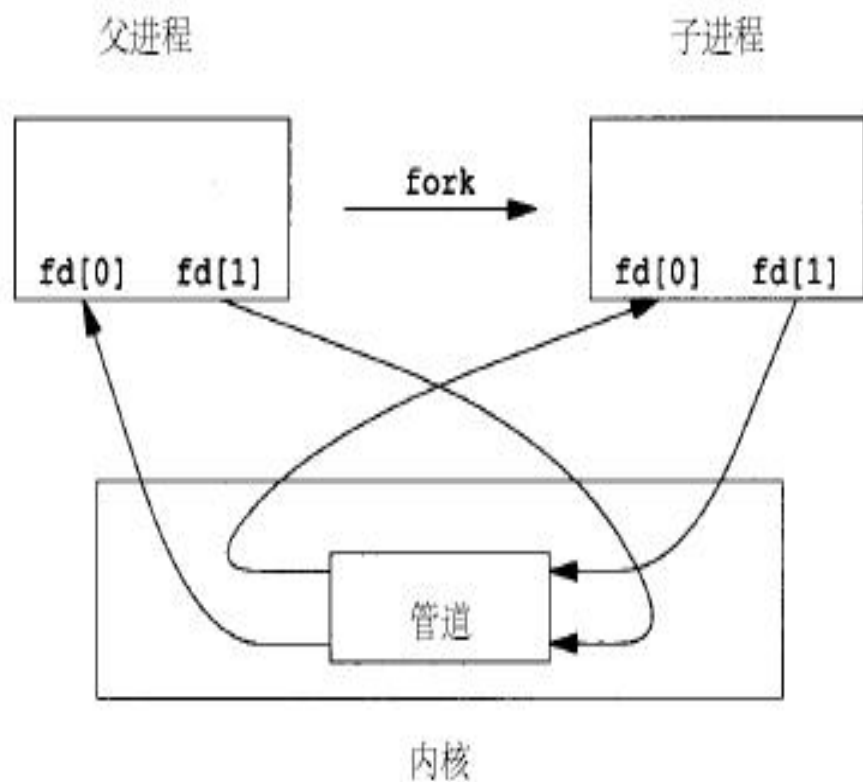


图14-2 fork之后的半双工管道

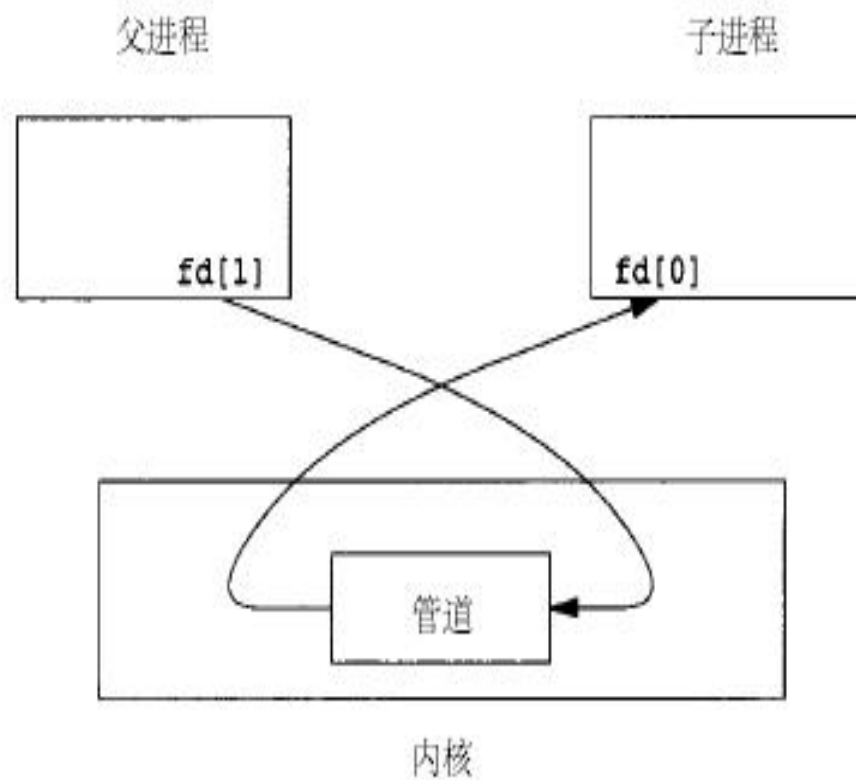
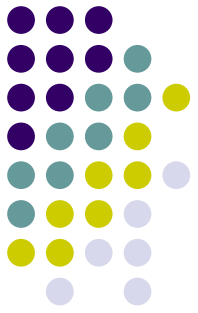
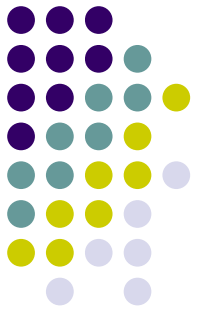


图14-3 从父进程到子进程的管道



# 管道使用(1): 单个进程

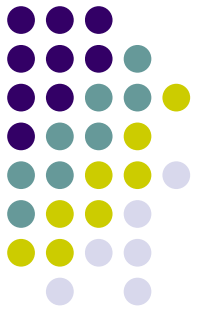
- 类似共享文件
- pipe—write——read



## 管道使用（2）多进程

- 父-子进程/子-子进程
- 使用模式：  
    pipe——fork——write（写进程）  
                    ——read（读进程）

注意： pipe-fork顺序



# 管道使用（3）用于标准输入输出

- 管道：shell中的形式

- cmd1 | cmd2
- 重定向 cmd > file

- 实现代码

- 执行cmd1前

```
if (fd[1] != STDOUT_FILENO) {  
    if (dup2(fd[1], STDOUT_FILENO) != STDOUT_FILENO)  
        err_sys("dup2 error to stdout);  
}
```

复制文件  
到stdout

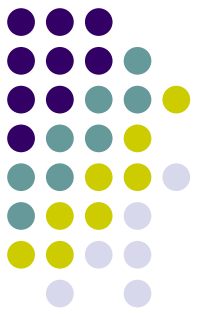
fd[1] 已经打开了, 先关闭, 再把fd[1]赋值给 stdout

标准输出

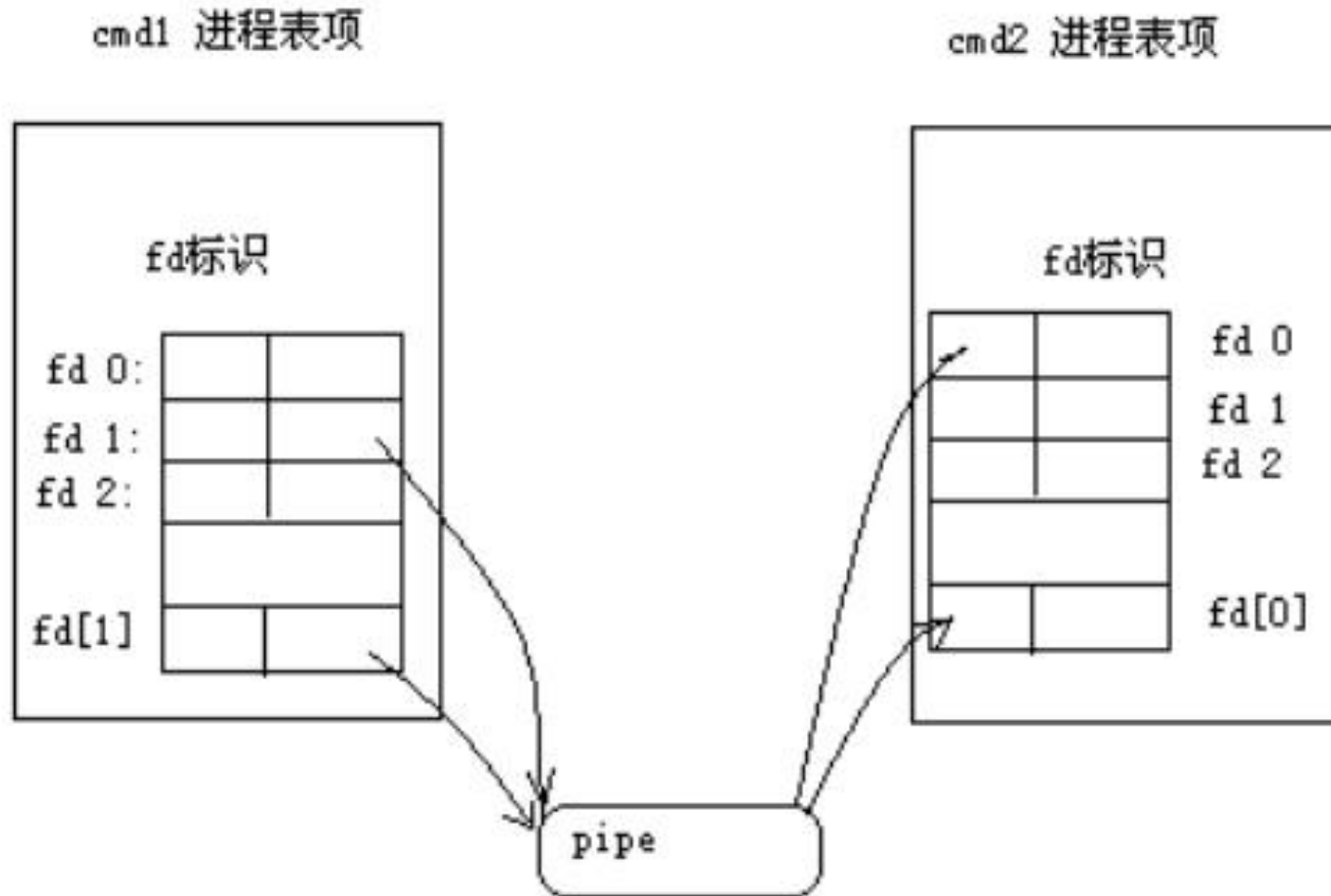
- 执行cmd2前

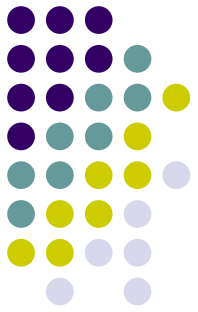
```
if (fd[0] != STDIN_FILENO) {  
    if (dup2(fd[0], STDIN_FILENO) != STDIN_FILENO)  
        err_sys("dup2 error to stdin);  
}
```

先  
把stdin关闭, 再把fd[0]赋值给 stdin



# 管道使用（3）用于标准输入输出





## 例5-4

利用管道实现父子进程

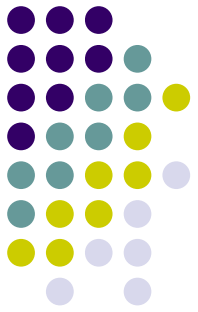
```
int main()
{
    int data_processed;  int file_pipes[2];
    const char some_data[] = "123";  pid_t fork_result;
    if (pipe(file_pipes) == 0)
    {
        fork_result = fork();
        if (fork_result == (pid_t)-1)
        {
            fprintf(stderr, "Fork failure");
            exit(EXIT_FAILURE);
        }
        if (fork_result == (pid_t)0)
        {
            close(0);
            dup(file_pipes[0]);
            close(file_pipes[0]);
            close(file_pipes[1]);
            execlp("od", "od", "-c", (char *)0);
            exit(EXIT_FAILURE);
        }
    }
}
```

**od -c file** 以字符方式显示文件内容，如果没指定文件则以标准输入作为默认输入

od -c file

子进程读

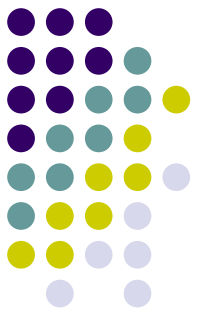




## 例5-4（续）

```
else
{
    close(file_pipes[0]);
    data_processed = write(file_pipes[1], some_data,
                           strlen(some_data));

    close(file_pipes[1]);
    printf("%d - wrote %d bytes\n", (int)getpid(),
           data_processed);
}
} exit(EXIT_SUCCESS);
}
```



## 2.3 popen & pclose 函数

- popen, pclose: process I/O

`#include <stdio.h>`

`FILE *popen(const char *command, const char *type);`

`int pclose(FILE *stream);`

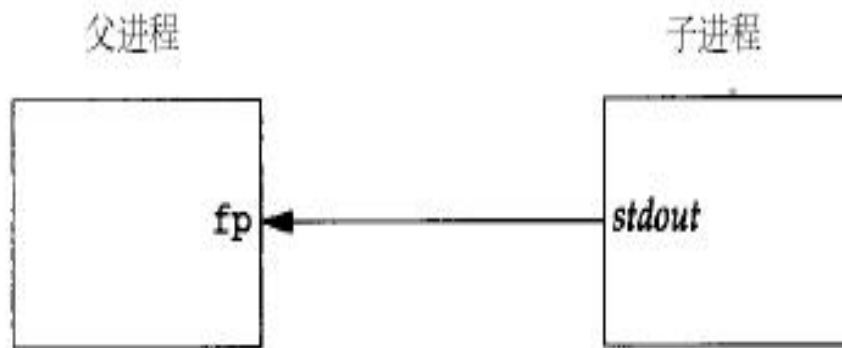


图14-5 `fp=popen(command, "r")`的结果

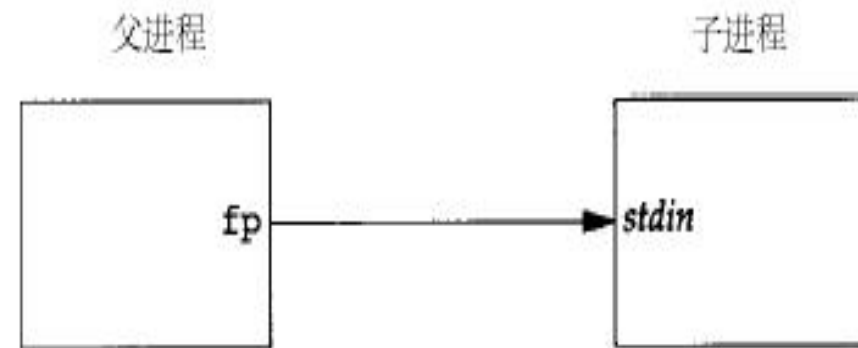
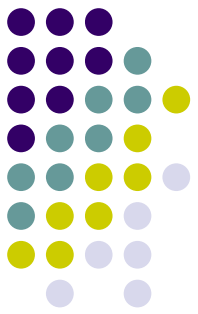


图14-6 `fp=popen(command, "w")`的结果



## 2.3 popen & pclose 函数

- **fp=popen(cmd,"r")**的pipe实现:

```
FILE * popen(char * cmd,char * type)
```

```
{ int fd; int p;
```

```
  pipe(fd);
```

```
  p=fork();
```

```
  if(p==0) { close(1);dup(fd[1]);  
             execl(cmd,cmd,NULL); }
```

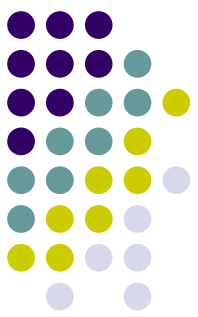
```
  else      { wait();
```

```
              fp=fdopen(fd[0],"r"); return fp; }
```

```
}
```

子进程执行命令

将低级文件描述符变为高级文件流指针。

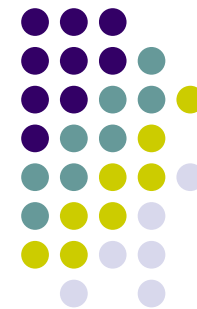


## 2.3 popen & pclose 函数

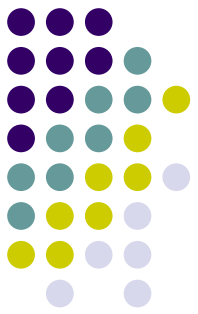
### popen 函数说明

|       |                                                                                                                                          |
|-------|------------------------------------------------------------------------------------------------------------------------------------------|
| 所需头文件 | <code>#include&lt;stdio.h&gt;</code>                                                                                                     |
| 函数功能  | 建立管道 I/O                                                                                                                                 |
| 函数原型  | <code>FILE * popen( const char * command,const char * type);</code>                                                                      |
| 函数传入值 | 调用 <code>fork()</code> 产生子进程, 然后从子进程中调用 <code>/bin/sh -c</code> 来执行参数 <code>command</code> 的指令。参数 <code>type</code> 可使用“r”代表读取, “w”代表写入。 |
| 函数返回值 | 若成功则返回文件指针, 否则返回 <code>NULL</code> , 错误原因存于 <code>errno</code> 中。                                                                        |
| 备注    | 在编写具 <code>SUID/SGID</code> 权限的程序时请尽量避免使用 <code>popen()</code> , <code>popen()</code> 会继承环境变量, 通过环境变量可能会造成系统安全的问题。                       |

## 例5-5

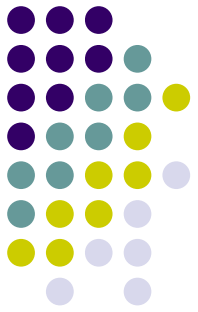


- 设计一程序，要求用popen创建管道，实现“ls -l|more”的功能。



```
int main ()
{  FILE *fp;    int num;          char buf[500];
  memset(buf,0,sizeof(buf));
  printf("建立管道.....\n");
  fp=popen("ls -l","r");
  if(fp!=NULL)
  {  num=fread(buf,sizeof(char),500,fp);
    if(num>0)
    { printf("第一个命令是 “ls -l”，运行结果如下： \n");
      printf("%s\n",buf);
    }
    pclose(fp);
  }
  else
  { printf("用popen创建管道错误\n");
    return 1;
  }
  fp=popen("grep -a fifo","w");
  printf("第二个命令是:grep fifo,");
  fwrite(buf,sizeof(char),500,fp);
  pclose(fp);
  return 0;
}
```

使用**popen**函数读写管道，实际上也是调用**pipe**函数建立一个管道，再调用**fork**函数建立子进程，接着会建立一个**shell**环境，并在这个**shell**环境中执行参数指定的进程。



# popen & pclose应用

- 在程序中获得另一个程序的输出

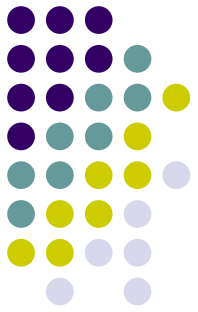
- 例5-6 popen\_two.c

```
gcc myucl.c -o myucl
```

```
gcc popen_two.c -o two
```

输入大写字母

按Ctrl+D结束程序



## 例5-6

程序头文件

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
```

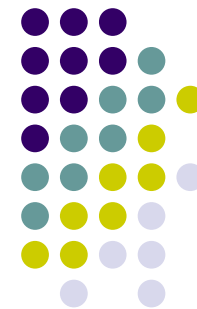
```
int main()
{
    char line[MAXLINE];    FILE *fpin;
    fpin = popen("./myucl", "r");
    if( NULL == fpin )      error_quit("popen error");
    while( 1 )
    {
        fputs("prompt> ", stdout);
        fflush(stdout);
        if( fgets(line, MAXLINE, fpin) == NULL )      break;
        if( fputs(line, stdout) == EOF ) perror("fputs error to pipe");
    }
    if( pclose(fpin) == -1 )      perror ("pclose error");
    putchar('\n');
    return 0;
}
```

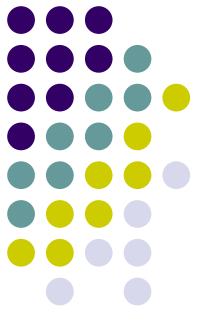


# 例5-6

## Myuc1c.c

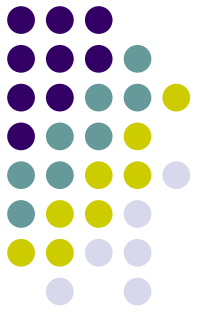
```
int main()
{
    int c;
    while( 1 )
    {
        c = getchar();
        if( EOF == c )
            break;
        if( isupper(c) )
            c = tolower(c);
        putchar(c);
        if( '\n' == c )
            fflush(stdout);
    }
    return 0;
}
```





## 2.4 FIFO: named pipe命名管道

- 管道和命名管道
  - 相同点
  - 不同点
    - 文件系统中
    - 内存传输数据
    - 同步：一个重要的考虑
  - `mkfifo(1)`, `mkfifo(3)`, `mknod(1)`, `mknod(2)`



# 创建FIFO

- 命令行方式: `mknod filename p`  
*创建设备文件*  
`mkfifo filename`
- 程序方式: `mkfifo`: make a FIFO special file (a named pipe)

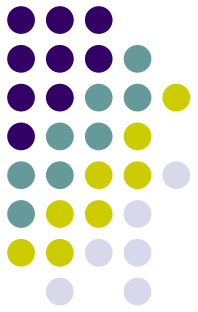
```
#include <sys/types.h>
```

```
#include <sys/stat.h>
```

```
int mkfifo(const char *pathname, mode_t mode);
```

(Returned value: 0 if success, -1 if failure)

例: `mkfifo("/tmp/myfifo", 0666)`



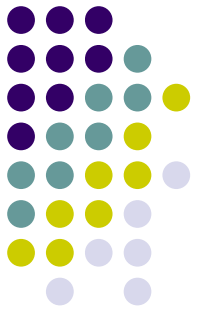
# 访问FIFO

- 命令行方式

(1) `cat < /tmp/myfifo` 读FIFO文件

(2) `echo hello >/tmp/myfifo` 向FIFO写数据

(3) `cat < /tmp/myfifo &`  
`echo "hello" >/tmp/myfifo`



# 程序方式：用open打开一个FIFO

- Review: “open” system call

int open(const char \*pathname, int flags);

- “flags” parameter

- 必须指定的互斥模式:

O\_RDONLY, O\_WRONLY, O\_NONBLOCK

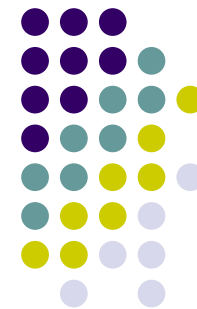
- O\_RDONLY: 若无进程写方式打开FIFO, open阻塞
- O\_RDONLY | O\_NONBLOCK: 若无进程写方式打开FIFO, open立即返回文件描述符 *不再阻塞*
- O\_WRONLY: 若无进程读方式打开FIFO, open阻塞
- O\_WRONLY | O\_NONBLOCK: 若无进程读方式打开FIFO, open返回ENXIO错误, -1

默认是阻塞的

## 例5-7

pipe 管道

fifo 先进先出结构



- 两个程序通过**FIFO**传递数据，一个生产者程序创建并打开一个**FIFO**，向管道中写入数据。  
（5-7fifo\_p.c）一个消费者程序，从**FIFO**中读取数据（5-7fifo\_c.c）。

```
#define FIFO_NAME "/tmp/my_fifo"
#define BUFFER_SIZE PIPE_BUF
#define TEN_MEG (1024 * 1024 * 10)
```

```
int main()
```

```
{ int pipe_fd; int res; int open_mode = O_WRONLY;
  int bytes_sent = 0; char buffer[BUFFER_SIZE];
  if (access(FIFO_NAME, F_OK) == -1)
  { res = mkfifo(FIFO_NAME, 0777);
    if (res != 0) { fprintf(stderr, "Could not create fifo\n");
      exit(EXIT_FAILURE); }
  }
```

int access(const char \*filepath, int mode)

R\_OK 只判断是否有读权限

W\_OK 只判断是否有写权限

X\_OK 判断是否有执行权限

F\_OK 只判断是否存在

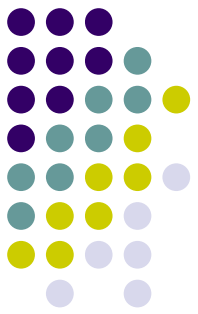
有效，则函数返回0，否则函数返回-1

```
    printf("Process %d opening FIFO O_WRONLY\n", getpid());
    pipe_fd = open(FIFO_NAME, open_mode);
    printf("Process %d result %d\n", getpid(), pipe_fd);
    if (pipe_fd != -1)
    { while(bytes_sent < TEN_MEG)
      { res = write(pipe_fd, buffer, BUFFER_SIZE);
        if (res == -1) { fprintf(stderr, "Write error on pipe\n"); exit(EXIT_FAILURE); }
        bytes_sent += res; }
      (void)close(pipe_fd); }
    else { exit(EXIT_FAILURE); }
    printf("Process %d finished\n", getpid());
    exit(EXIT_SUCCESS);
}
```

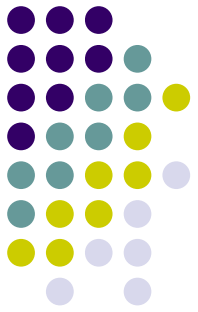
```

#define FIFO_NAME "/tmp/my_fifo"
#define BUFFER_SIZE PIPE_BUF
int main()
{
    int pipe_fd;    int res;    int open_mode = O_RDONLY;    char
    buffer[BUFFER_SIZE + 1];
    int bytes_read = 0;
    memset(buffer, '\0', sizeof(buffer));
    printf("Process %d opening FIFO O_RDONLY\n", getpid());
    pipe_fd = open(FIFO_NAME, open_mode);
    printf("Process %d result %d\n", getpid(), pipe_fd);
    if (pipe_fd != -1) {
        do {
            res = read(pipe_fd, buffer, BUFFER_SIZE);
            bytes_read += res;
        } while (res > 0);
        (void)close(pipe_fd);
    }
    else {
        exit(EXIT_FAILURE);    }
    printf("Process %d finished, %d bytes read\n", getpid(),
        bytes_read);
    exit(EXIT_SUCCESS);
}

```







# FIFO的应用(1)

- 用FIFO复制输出流
  - 例

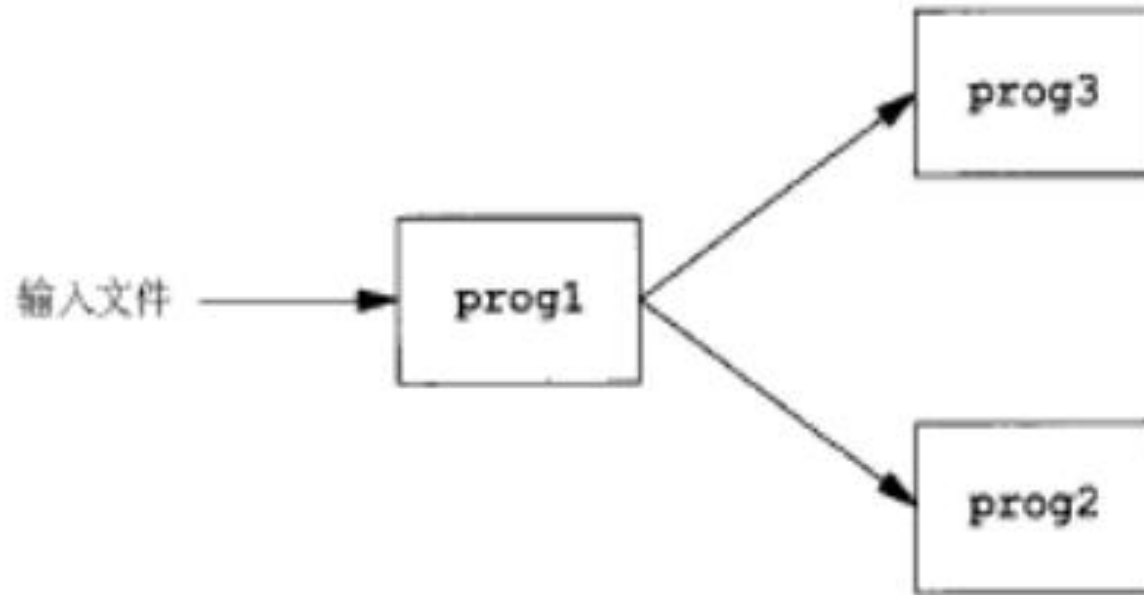


图14-9 对一个经过过滤的输入流进行两次处理

# FIFO的应用(1)

```
mkfifo fifo1  
prog3 < fifo1 &  
prog1 < infile | tee fifo1 | prog2
```

tee命令读取标准输入，把这些内容同时输出到标准输出和（多个）文件中

可执行

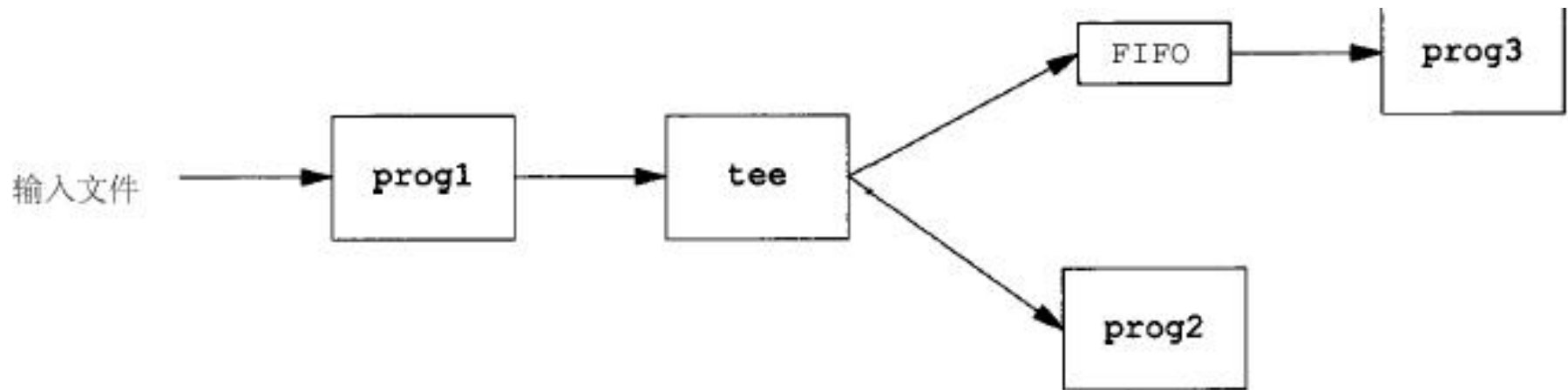


图14-10 使用FIFO和tee将一个流发送到两个不同的进程

# FIFO的应用(2)

- C/S应用程序
  - 例5-8 [client.c](#), [server.c](#)

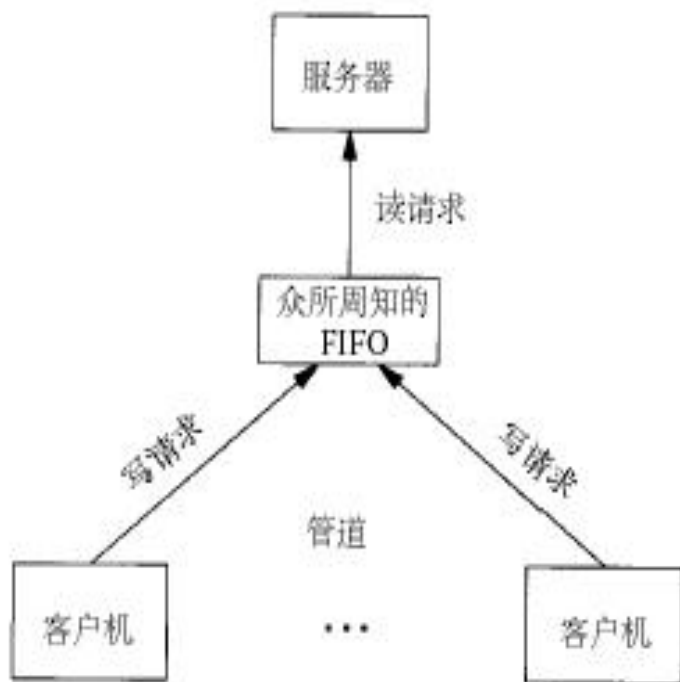


图14-11 客户机用FIFO向服务器发送请求

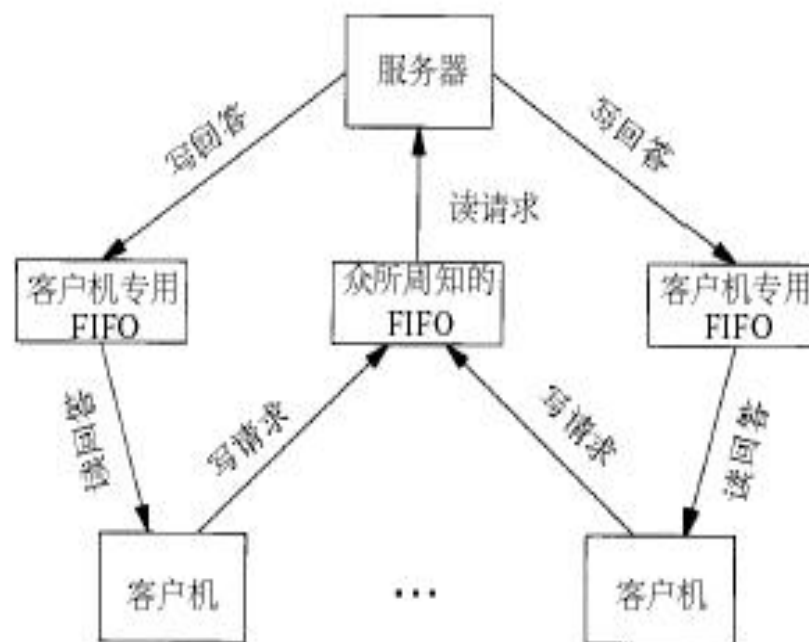
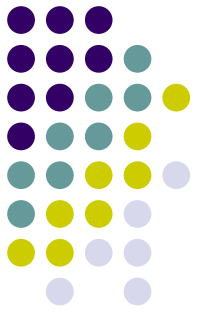


图14-12 客户机-服务器用FIFO进行通信

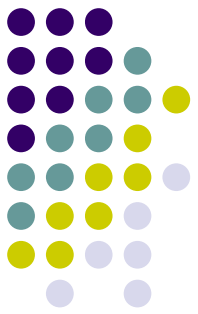
# Client.h



```
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <limits.h>
#include <sys/types.h>
#include <sys/stat.h>
#define SERVER_FIFO_NAME "/tmp/serv_fifo"
#define CLIENT_FIFO_NAME "/tmp/cli_%d_fifo"
#define BUFFER_SIZE 20
struct data_to_pass_st {
    pid_t client_pid;
    char  some_data[BUFFER_SIZE - 1];
};
```

int main()   // 客户端程序

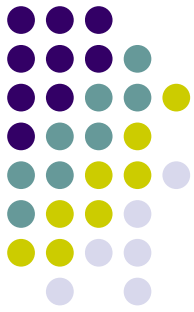
```
{  int server_fifo_fd, client_fifo_fd;  struct data_to_pass_st my_data;
  int times_to_send;  char client_fifo[256];
  server_fifo_fd = open(SERVER_FIFO_NAME, O_WRONLY);
  if (server_fifo_fd == -1) {
    fprintf(stderr, "Sorry, no server\n");
    exit(EXIT_FAILURE);  }
  my_data.client_pid = getpid();
  sprintf(client_fifo, CLIENT_FIFO_NAME, my_data.client_pid);
  if (mkfifo(client_fifo, 0777) == -1) {
    fprintf(stderr, "Sorry, can't make %s\n", client_fifo);
    exit(EXIT_FAILURE);  }
  for (times_to_send = 0; times_to_send < 5; times_to_send++) {
    sprintf(my_data.some_data, "Hello from %d", my_data.client_pid);
    printf("%d sent %s, ", my_data.client_pid, my_data.some_data);
    write(server_fifo_fd, &my_data, sizeof(my_data));
    client_fifo_fd = open(client_fifo, O_RDONLY);
    if (client_fifo_fd != -1) {
      if (read(client_fifo_fd, &my_data, sizeof(my_data)) > 0) {
        printf("received: %s\n", my_data.some_data);
      }
      close(client_fifo_fd);  }  }
  close(server_fifo_fd);
  unlink(client_fifo);
  exit(EXIT_SUCCESS);}
```

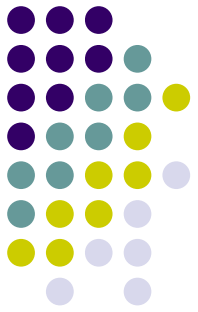


int main() **//服务端程序**

```
{  int server_fifo_fd, client_fifo_fd;  struct data_to_pass_st my_data;  int read_res;  char client_fifo[256];  char *tmp_char_ptr;

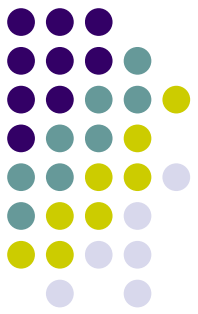
  mkfifo(SERVER_FIFO_NAME, 0777);
  server_fifo_fd = open(SERVER_FIFO_NAME, O_RDONLY);
  if (server_fifo_fd == -1) {
    fprintf(stderr, "Server fifo failure\n");
    exit(EXIT_FAILURE);  }
  sleep(10); /* lets clients queue for demo purposes */
  do {  read_res = read(server_fifo_fd, &my_data, sizeof(my_data));
    if (read_res > 0) {
      tmp_char_ptr = my_data.some_data;
      while (*tmp_char_ptr) {
        *tmp_char_ptr = toupper(*tmp_char_ptr);
        tmp_char_ptr++;      }
      sprintf(client_fifo, CLIENT_FIFO_NAME, my_data.client_pid);
      client_fifo_fd = open(client_fifo, O_WRONLY);
      if (client_fifo_fd != -1) {
        write(client_fifo_fd, &my_data, sizeof(my_data));
        close(client_fifo_fd);  }
    }  } while (read_res > 0);
  close(server_fifo_fd);
  unlink(SERVER_FIFO_NAME);
  exit(EXIT_SUCCESS);}
```





## 例5-9

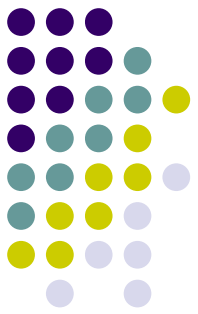
- 设计两个程序，要求用命名管道**FIFO**实现简单的聊天功能。
- Zhang.c
- Li.c



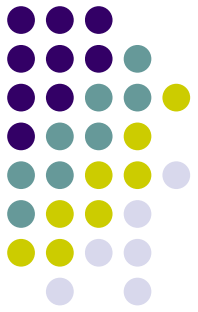
```
int main()
{
    int i, rfd,wfd,len=0,fd_in;  char str[32];  int flag,stdinflag;  fd_set
    write_fd,read_fd;  struct timeval net_timer;
    mkfifo("fifo1",S_IWUSR|S_IRUSR|S_IRGRP|S_IROTH);
    mkfifo("fifo2",S_IWUSR|S_IRUSR|S_IRGRP|S_IROTH);
    wfd =open("fifo1",O_WRONLY);  rfd =open("fifo2",O_RDONLY);
    if(rfd<=0||wfd<=0)return 0;  printf( "这是张三端!\n");
    while(1)
    {
        FD_ZERO(&read_fd); 初始化
        FD_SET(rfd,&read_fd); rfd 接收 read_fd
        FD_SET(filenno(stdin),&read_fd); /**/
        net_timer.tv_sec=5; 5秒超时
        net_timer.tv_usec=0;
        memset(str,0,sizeof(str));
        if(i=select(rfd+1, &read_fd,NULL, NULL, &net_timer) <= 0)
        continue;
        if(FD_ISSET(rfd,&read_fd))
        {
            read(rfd,str,sizeof(str));  printf("-----\n");
            printf("李四:%s\n",str);  }
        if(FD_ISSET(filenno(stdin),&read_fd))
        {
            printf("-----\n");  fgets(str,sizeof(str),stdin);
            len=write(wfd,str,strlen(str));
        }
    }
    close(rfd);
    close(wfd);}

```



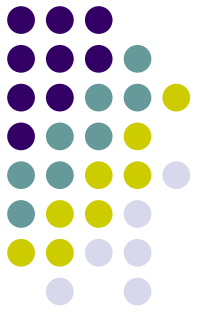


```
int main()
{   int i, rfd,wfd,len=0,fd_in;           char str[32];           int flag,stdinflag;
    fd_set write_fd,read_fd;struct timeval net_timer;
    mkfifo("fifo1",S_IWUSR|S_IRUSR|S_IRGRP|S_IROTH);
    mkfifo("fifo2",S_IWUSR|S_IRUSR|S_IRGRP|S_IROTH);
    rfd =open("fifo1",O_RDONLY);   wfd =open("fifo2",O_WRONLY);
    if(rfd<=0||wfd<=0)return 0;           printf("这是李四端!\n");
    while(1)
    { FD_ZERO(&read_fd);                 FD_SET(rfd,&read_fd);
      FD_SET(fileno(stdin),&read_fd);           net_timer.tv_sec=5;
      net_timer.tv_usec=0;                     memset(str,0,sizeof(str));
      if(i=select(rfd+1,&read_fd,NULL, NULL, &net_timer) <= 0) continue;
          if(FD_ISSET(rfd,&read_fd))
          {
              read(rfd,str,sizeof(str));
              printf("-----\n");
              printf("张三:%s\n",str);           }
          if(FD_ISSET(fileno(stdin),&read_fd))
          {
              printf("-----\n");
              fgets(str,sizeof(str),stdin);
              len=write(wfd,str,strlen(str)); /*写入管道*       }
    }      close(rfd);
    close(wfd);
}
```



# 掌握的知识点

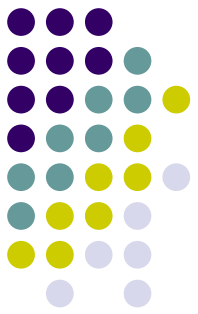
- 管道两种实现方式的异同
- `popen`函数的实现及应用
- FIFO的原理和使用方法
- C-S通信程序设计



# Part 3 System V IPC

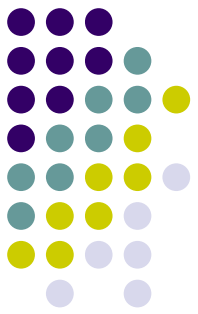
研究网络

- IPC objects
  - 信号量(semaphore set)
  - 消息队列(message queue)
  - 共享内存(shared memory)
- shell命令
  - `ipcs -q/-m/-s`
  - `ipcrm -q/-m/-s <id>`
  - `ipcrm -Q/-M/-S <key>`



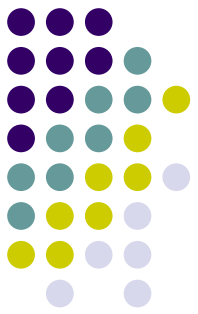
## 3.1 System V IPC的共同特征

- 标识符与关键字
  - 创建IPC对象时指定关键字(key\_t key;)
    - key的选择; 预定义常数IPC\_PRIVATE; ftok函数
  - 引用IPC对象: 标识符
  - 内核将关键字转换成标识符
- 许可权结构
  - 和文件类比
  - struct ipc\_perm



# SV IPC System Calls Overview

| 功能                             | 消息队列              | 信号量           | 共享内存                    |
|--------------------------------|-------------------|---------------|-------------------------|
| 分配一个IPC对象，获得对IPC的访问            | <u>msgget</u>     | <u>semget</u> | <u>shmget</u>           |
| IPC操作: 发送/接收消息，信号量操作，连接/释放共享内存 | msgsnd/<br>msgrcv | semop         | <u>shmat</u> /<br>shmdt |
| IPC控制: 获得/修改状态信息，取消IPC         | msgctl            | semctl        | shmctl                  |



# ftok函数

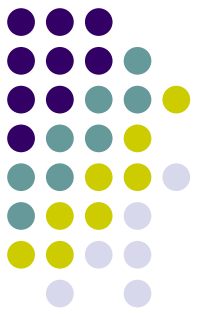
## 创建函数

```
key_t ftok( char * filename, int id);
```

## 功能说明

将一个已存在的文件名(该文件必须是存在而且可以访问的)和一个整数标识符id转换成一个key\_t值

在Linux系统实现中，调用该函数时，系统将文件的索引节点号取出，并在前面加上子序号，从而得到key\_t的返回值

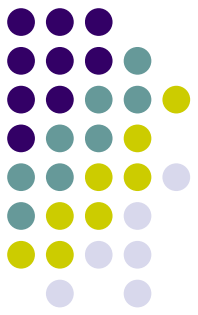


# 创建IPC对象

- key: 可由ftok()函数产生或定义为IPC\_PRIVATE常量
- flag: 包括读写权限, 还可包含IPC\_CREATE和IPC\_EXCL标志位, 组合效果如下

| oflag标志            | IPC对象不存在            | IPC对象已存在         |
|--------------------|---------------------|------------------|
| 无特殊标志              | 出错,<br>errno=ENOENT | 成功, 引用已存在对象      |
| IPC_CREAT          | 成功, 创建新对象           | 成功, 引用已存在对象      |
| IPC_CREAT IPC_EXCL | 成功, 创建新对象           | 出错, errno=EEXIST |

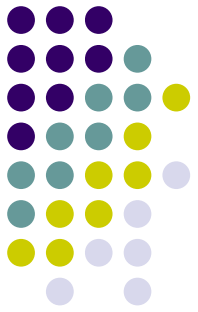
不存在  
互斥  
存在



## 3.2 Message queue

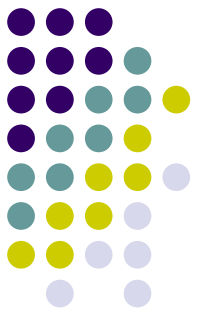
- 消息队列
  - 消息队列是消息的链表，存放在内核中并由消息队列标识符标识。
  - First in, first out
  - message type: 优先级
  - 块数据





# 消息队列——程序结构

- proto.h:约定的消息队列通信格式
- send.c
- receive.c



# 消息队列——程序结构

- proto.h:约定的消息队列通信格式:

```
#define KEYPATH "/etc/services"
```

```
#define KEYPROJ 'a'
```

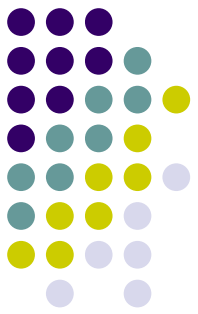
```
struct msg_st {
```

```
    char * msg;
```

```
    long  type;
```

```
    ...
```

```
}
```



# 消息队列——系统函数

```
#include <sys/types.h>
```

```
#include <sys/ipc.h>
```

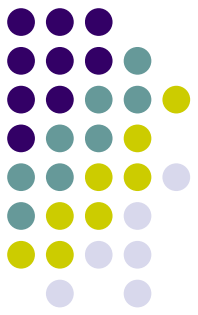
```
#include <sys/msg.h>
```

```
int msgget(key_t key, int flag);
```

```
int msgsnd(int msqid, const void *ptr, size_t nbytes,  
            int flag);
```

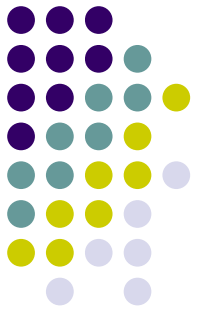
```
int msgrcv(int msqid, void *ptr, size_t nbytes, long  
            type, int flag);
```

```
int msgctl(int msqid, int cmd, struct shmid_ds *buf);
```



# msgget

- 函数原型 `int msgget(key_t key, int flag);`
- 参数说明
  - ✓ **key:** 待创建/打开队列的键值
  - ✓ **flag:** 创建/打开方式
    - 常取 `IPC_CREAT|IPC_EXCL|0666`
    - 若不存在**key**值的队列，则创建；否则，若存在，则打开队列
    - 0666**表示与一般文件权限一样
- 返回值
  - 成功返回消息队列描述字，否则返回-1
- 说明
  - IPC\_CREAT**一般由服务器程序创建消息队列时使用
  - 若是客户程序，须打开现有消息队列，而不用**IPC\_CREAT**



# msgsnd

## ➤ 函数原型

```
int msgsnd(int msqid, struct msgbuf *msgp, size_t size, int flag);
```

## ➤ 说明

**flag**有意义的标志为IPC\_NOWAIT，指明在消息队列没有足够空间容纳要发送的消息时，msgsnd是否等待

## ➤ 内核须对msgsnd( )函数完成的工作

### ✓ 检查消息队列描述符、许可权及消息长度

若合法，继续执行

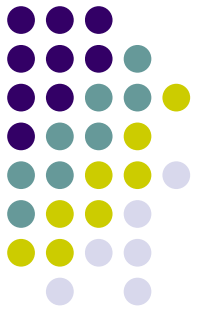
否则，返回

### ✓ 内核为消息分配数据区，将消息正文拷贝到消息数据区

### ✓ 分配消息首部，并将它链入消息队列的末尾

### ✓ 修改消息队列头数据，如队列中的消息数、字节总数等

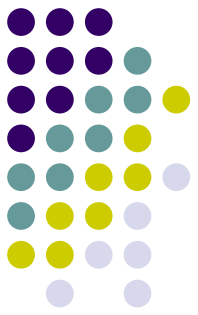
### ✓ 唤醒等待消息的进程



# msgrcv

- `int msgrcv(int msqid, void *ptr, size_t nbytes, long type, int flag);`
- 参数说明
  - `msqid`: 消息队列描述字
  - `msgp`: 消息存储位置
  - `size`: 消息内容的长度
  - `type`: 请求读取的消息类型
  - `flag`: 规定队列无消息时内核应做的操作
    - `IPC_NOWAIT`: 无满足条件消息时返回，此时 `errno=ENOMSG`
    - `IPC_EXCEPT`: `type>0`时使用，返回第一个类型不为`type`的消息
    - `IPC_NOERROR`: 如果队列中满足条件的消息内容大于所请求的`size`字节，则把该消息截断，截断部分将丢失

# msgctl: message control operations

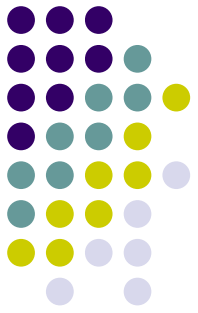


- 函数原型

`int msgctl(int msqid, int cmd, struct msqid_ds *buf);`

- “cmd” 参数

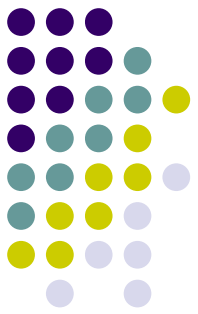
- IPC\_STAT: 把msqid\_ds结构中的数据置为消息队列的当前关联值
- IPC\_SET: 在进程有足够权限的前提下，把消息队列的当前关联值置为msqid\_ds结构给出的值
- IPC\_RMID: 删除消息队列



# 消息队列——Example

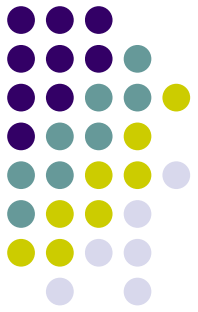
- A C/S application
  - One server, several clients: only one queue is required.
  - Compared with FIFO implementation





# 消息队列属性

- 消息队列创建后，操作系统在内核中分配了一个名称为**msqid\_ds**的数据结构用于管理该消息队列。
- 在程序中可以通过函数**msgctl**对该结构进行读写，从而实现对消息队列的控制功能。



# 消息队列属性

## 成员说明:

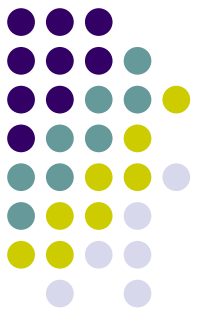
- 1) `msg_perm`: IPC许可权限结构。
- 2) `msg_stime`: 最后一次向该消息队列发送消息 (`msgsnd`) 的时间。
- 3) `msg_rtime`: 最后一次从该消息队列接收消息 (`msgrcv`) 的时间。
- 4) `msg_ctime`: 最后一次调用`msgctl`的时间。
- 5) `msg_cbytes`: 当前该消息队列中的消息长度, 以字节为单位。
- 6) `msg_qnum`: 当前该消息队列中的消息条数。
- 7) `msg_qbytes`: 该消息队列允许存储的最大长度, 以字节为单位。
- 8) `msg_lspid`: 最后一次向该消息队列发送消息 (`msgsnd`) 的进程ID。
- 9) `msg_lrpid`: 最后一次从该消息队列接收消息 (`msgrcv`) 的进程ID。

## 使用:

```
msg_sinfo.msg_qbytes = 1666;  
msgctl(msgqid,IPC_SET,&msg_sinfo)
```

# 消息队列的属性

## 例5-10 msg\_stat.c



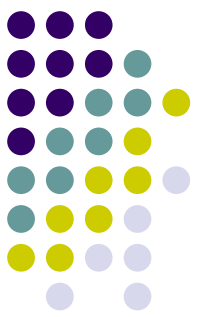
```
struct msgbuf{
    int mtype;
    char mtext[1];
}msg_sbuf;
struct msgmbuf {
    int mtype;
    char mtext[10];
}msg_rbuf;

main( ){
    int gflags,sflags,rflags;
    key_t key;
    int msgid;
    int reval;

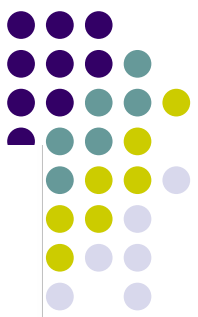
    struct msqid_ds msg_ginfo,msg_sinfo;
```



```
/*创建一个消息队列,输出消息队列缺省属性*/
char* msgpath="/UNIX/msgqueue";
key=ftok(msgpath, 'a');
gflags=IPC_CREAT|IPC_EXCL;
msgid=msgget(key,gflags|00666);
if(msgid==-1) {
    printf("msg create error\n");
    return;
}
msg_stat(msgid,msg_ginfo);
sflags=IPC_NOWAIT;
msg_sbuf.mtype=10;
msg_sbuf.mtext[0]='a';
```



```
/*发送一个消息,输出消息队列属性*/
reval=msgsnd(msgid,&msg_sbuf,sizeof(msg_sbuf.mtext),sflags);
if(reval== -1) {
    printf("message send error\n");
}
msg_stat(msgid,msg_ginfo);
rflags=IPC_NOWAIT|MSG_NOERROR;
reval=msgrcv(msgid,&msg_rbuf,4,10,rflags);
if(reval== -1)
    printf("read msg error\n");
else
    printf("read from msg queue %d bytes\n",reval);
```

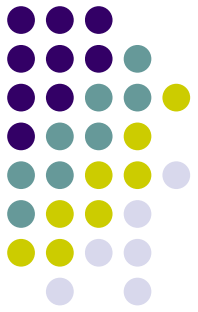


```
/*从消息队列中读出消息后，输出消息队列属性*/
msg_stat(msgid,msg_ginfo);
msg_sinfo.msg_perm.uid=8;
msg_sinfo.msg_perm.gid=8;
msg_sinfo.msg_qbytes=16388;
/*此处验证超级用户可以更改消息队列的缺省msg_qbytes*/
/*注意这里设置的值大于缺省值*/
reval=msgctl(msgid,IPC_SET,&msg_sinfo);
if(reval== -1) {
    printf("msg set info error\n");
    return;
}
msg_stat(msgid,msg_ginfo);
/*验证设置消息队列属性*/
reval=msgctl(msgid,IPC_RMID,NULL);/*删除消息队列*/
if(reval== -1) {
    printf("unlink msg queue error\n");
    return;
}
```



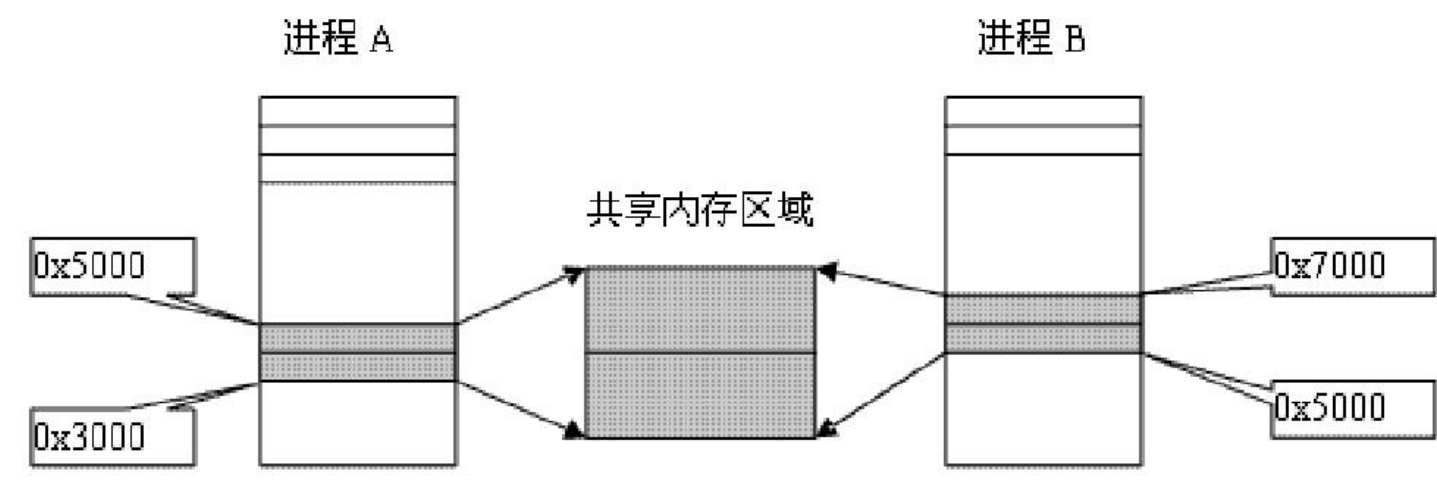


```
void msg_stat(int msgid, struct msqid_ds msg_info) {
    int reval;
    sleep(1); /*只是为了后面输出时间的方便*/
    reval=msgctl(msgid, IPC_STAT, &msg_info);
    if(reval== -1) {
        printf("get msg info error\n");
        return;
    }
    printf("\n");
    printf("current number of bytes on queue is %d\n", msg_info.msg_cbytes);
    printf("number of messages in queue is %d\n", msg_info.msg_qnum);
    printf("max number of bytes on queue is %d\n", msg_info.msg_qbytes);
    /*每个消息队列的容量（字节数）都有限制MSGMNB, 值的大小因系统而异*/
    /*在创建新的消息队列时, msg_qbytes的缺省值就是MSGMNB*/
    printf("pid of last msgsnd is %d\n", msg_info.msg_lspid);
    printf("pid of last msgrcv is %d\n", msg_info.msg_lrpid);
    printf("last msgsnd time is %s", ctime(&(msg_info.msg_stime)));
    printf("last msgrcv time is %s", ctime(&(msg_info.msg_rtime)));
    printf("last change time is %s", ctime(&(msg_info.msg_ctime)));
    printf("msg uid is %d\n", msg_info.msg_perm.uid);
    printf("msg gid is %d\n", msg_info.msg_perm.gid);
}
```

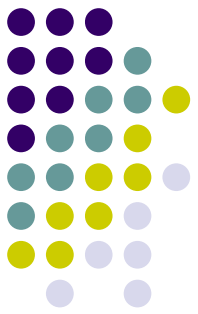


## 3.3 Shared memory

- 共享内存
  - 共享内存是内核为进程创建的一个特殊内存段，它可连接(**attach**)到自己的地址空间，也可以连接到其它进程的地址空间
  - 最快的进程间通信方式
  - 不提供任何同步功能







# 共享内存实现途径

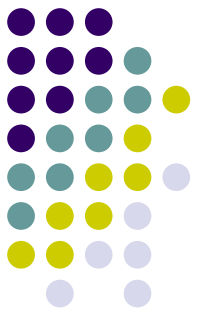
- **mmap()**系统调用

将普通文件在不同的进程中打开并映射到内存  
不同进程通过访问映射来达到共享内存目的

- **POSIX共享内存机制（Linux 2.6未实现）**

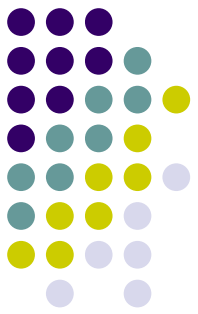
- **System V共享内存**

在内存文件系统**tmpfs**中建立文件  
文件映射到不同进程空间



# 共享内存（1）

- **mmap()**系统调用使得进程之间通过映射同一个普通文件实现共享内存。普通文件被映射到进程地址空间后，进程可以像访问普通内存一样对文件进行访问，不必再调用**read()**，**write()**等操作。



# mmap()共享内存

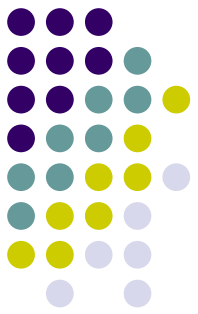
```
#include <unistd.h >
```

```
#include <sys/mman.h >
```

```
void* mmap ( void * addr , size_t len , int prot , int  
            flags , int fd , off_t offset )
```

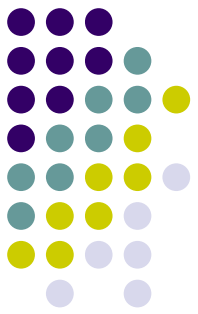
```
int munmap( void * addr, size_t len )
```

```
int msync ( void * addr , size_t len, int flags)
```



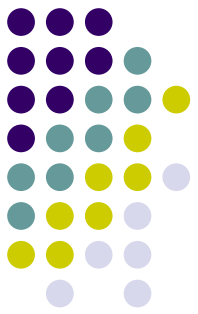
# 相关函数（1） mmap

- `void* mmap ( void * addr , 映射长度 size_t len , int prot , int flags , int fd , off_t offset )`
- **addr**:指定文件应被映射到进程空间的起始地址，一般被指定一个空指针，此时选择起始地址的任务留给内核来完成。
- **len**:映射到调用进程地址空间的字节数，从被映射文件开头offset个字节开始算起。
- **prot**:指定共享内存的访问权限。可取如下几个值的或：  
**PROT\_READ**（可读），**PROT\_WRITE**（可写），**PROT\_EXEC**（可执行），**PROT\_NONE**（不可访问）。



# 相关函数（1） mmap

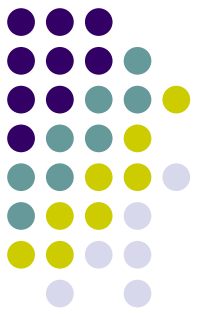
- **flags**;指定映射对象的类型，映射选项和映射页是否可以共享。由以下几个常值指定：**MAP\_SHARED**，**MAP\_PRIVATE** 必选其一。
- **fd**:为即将映射到进程空间的文件描述字，一般由**open()**返回，同时，**fd**可以指定为-1，此时须指定**flags**参数中的**MAP\_ANONYMOUS**，表明进行的是匿名映射（不涉及具体的文件名，避免了文件的创建及打开，只能用于具有亲缘关系的进程间通信）。
- **offset**参数一般设为0，表示从文件头开始映射。
- 函数的返回值为最后文件映射到进程空间的地址，进程可直接操作起始地址为该值的有效地址。



# 相关函数（1） mmap

- mmap()用于共享内存的两种方式：
  - （1）使用普通文件提供的内存映射：适用于任何进程之间；需要打开或创建一个文件，然后再调用mmap()；调用代码如下：

```
fd=open(name, flag, mode);  
if(fd>0) 分配块内存地址  
ptr=mmap(NULL, len , PROT_READ|PROT_WRITE,  
MAP_SHARED , fd , 0);
```



# 相关函数（1） mmap

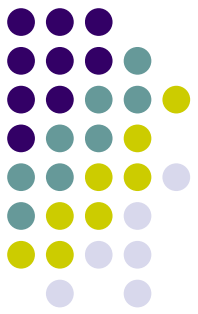
（2） 使用特殊文件提供匿名内存映射：适用于具有亲缘关系的进程之间；

- 调用代码如下：

```
ptr=mmap(NULL, len , PROT_READ|PROT_WRITE,  
MAP_SHARED|MAP_ANONYMOUS ,-1 , 0);
```

- 由于父子进程特殊的亲缘关系，在父进程中先调用  
mmap()，然后调用fork()。在调用fork()之后，子进程继承父进程匿名映射后的地址空间，也继承mmap()返回的地址

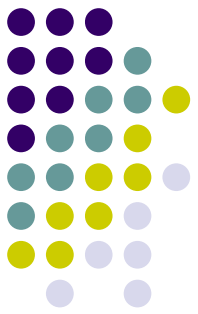
不必指定具体的文件，只要设置相应的标志即可



## 相关函数（2） **munmap**

- `int munmap( void * addr, size_t len )`  
该调用在进程地址空间中解除一个映射关系，**addr**是调用**mmap()**时返回的地址，**len**是映射区的大小。当映射关系解除后，对原来映射地址的访问将导致段错误发生。



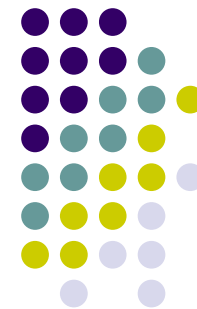


## 相关函数（3） **msync()**

**int msync ( void \* addr , size\_t len, int flags)**

一般说来，进程在映射空间的对共享内容的改变并不直接写回到磁盘文件中，往往在调用 **munmap ( )** 后才执行该操作。可以通过调用 **msync()** 实现磁盘上文件内容与共享内存区的内容一致。

## 例5-11 mmap.c



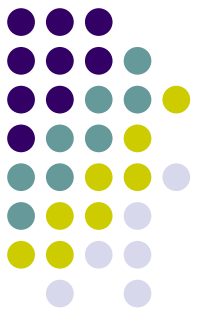
- 设计一个程序，要求复制进程，父子进程通过匿名映射实现共享内存。

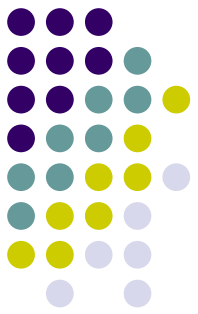
```

typedef struct
{   char name[4];
    int  age;
}people;

main(int argc, char** argv)
{   pid_t result;          int i;    people *p_map;          char temp;
    p_map=(people*)mmap(NULL,sizeof(people)*10,PROT_READ|P
ROT_WRITE,MAP_SHARED|MAP_ANONYMOUS,-1,0);
    result=fork();
    if(result<0)
    {   perror("创建子进程失败");      exit(0); }
    else if (result==0)
    {   sleep(2);
        for(i = 0;i<5;i++)
            printf("子进程读取: 第 %d 个人的年龄是:
%d\n",i+1,(*(p_map+i)).age);
            (*p_map).age = 110;
            munmap(p_map,sizeof(people)*10); /*解除内存映射关系*/
            exit(0);
    }
}

```

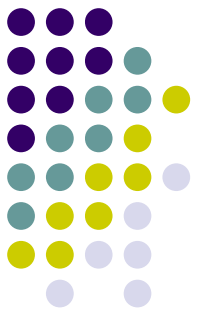




```
else {  
    temp = 'a';  
    for(i = 0;i<5;i++)  
    {  
        temp += 1;  
        memcpy((*p_map+i).name, &temp,1);  
        (*p_map+i).age=20+i;  
    }  
    sleep(5);  
    printf( "父进程读取: 五个人的年龄和是:  
%d\n",(*p_map).age );  
    printf("解除内存映射.....\n");  
    munmap(p_map,sizeof(*p_map));  
    printf("解除内存映射成功\n");  
}
```

解除内存映射

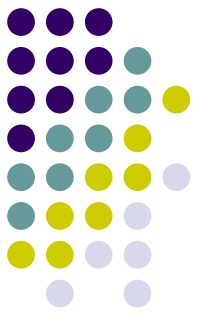
使用特殊文件提供匿名内存映射，适用于具有亲缘关系的进程之间。一般而言，子进程单独维护从父进程继承下来的一些变量。而 **mmap** 函数的返回地址，由父子进程共同维护。



# System V IPC共享内存的实现

- 通过映射到tmpfs中的shm文件对象实现共享主存  
每个共享主存区对应tmpfs中的一个文件（通过shmid\_kernel关联）
- 创建过程  
从主存申请共享主存管理结构，初始化相应shmid\_kernel结构  
在tmpfs中创建并打开一个同名文件  
在主存中建立与该文件对应的dentry及inode结构  
返回相应标识符

# 共享内存（2） System V shared memory



```
#include <sys/types.h>
```

```
#include <sys/ipc.h>
```

```
#include <sys/shm.h>
```

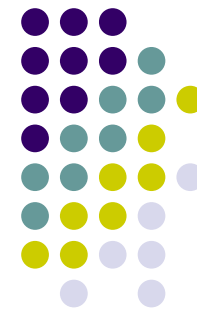
```
int shmget(key_t key, int size, int flag);
```

```
void *shmat(int shmid, void *addr, int flag);
```

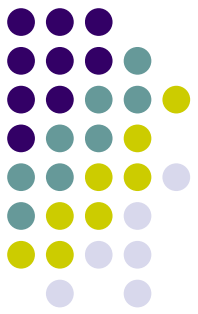
```
int shmdt(void *addr);
```

```
int shmctl(int shmid, int cmd, struct shmid_ds *buf);
```

# mmap和System V



1. System V共享内存中的数据，从来不写入到实际磁盘文件中；而通过mmap()映射普通文件实现的共享内存通信可以指定何时将数据写入磁盘文件中。
2. System V共享内存是随内核持续的，即使所有访问共享内存的进程都已经正常终止，共享内存区仍然存在（除非显式删除共享内存），在内核重新引导之前，对该共享内存区域的任何改写操作都将一直保留。



## 3.4 信号量

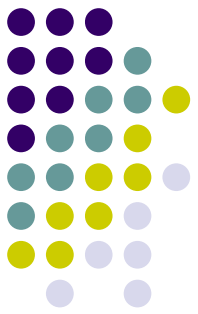
- 用来协调进程对共享资源的访问
- 相关函数 **semget**, **semop**, **semctl**
- 所需头文件

```
#include <sys/types.h>
```

```
#include <sys/ipc.h>
```

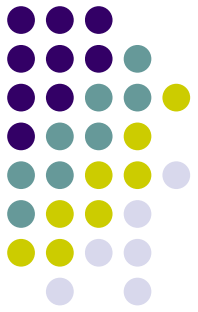
```
#include <sys/sem.h>
```





# 相关函数——semget

- 创建一个新信号量或取得一个已有信号量，原型为：  
`int semget(key_t key, int num_sems, int sem_flags);`
- 参数**key**：整数值
- 参数**num\_sems**：指定需要的信号量数目，几乎总是1。
- 参数**sem\_flags**：一组标志，信号量不存在时创建一个新的信号量，指定`IPC_CREAT`做按位或操作。设置了`IPC_CREAT`标志后，即使给出的键是一个已有信号量的键，也不会产生错误。  
指定`IPC_CREAT | IPC_EXCL`，创建一个新的，唯一的信号量，如果信号量已存在，返回一个错误。
- 返回值：成功返回一个相应信号标识符（非零），失败返回-1。



# 相关函数——semop

操作一个或一组信号，原型为：

`int semop(int sem_id, struct sembuf *sem_opa, size_t nsops);`

- **semid**: 信号集的识别码，可通过[semget](#)获取。
- **sem\_opa**: 指向存储信号操作结构的[数组指针](#)，信号操作结构的原型如下

**struct sembuf**{

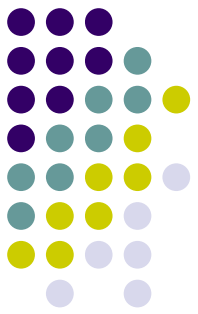
**short sem\_num**; //信号量集中的信号量编号0，1.....

**short sem\_op**; //信号量在一次操作中需要改变的数据，通常是两个数，一个是-1，一个是+1。

**short sem\_flg**; //通常为SEM\_UNDO,使操作系统跟踪信号并在进程没有释放该信号量而终止时，操作系统释放信号量

};

- **nsops**: 信号量操作结构的数量，大于或等于1



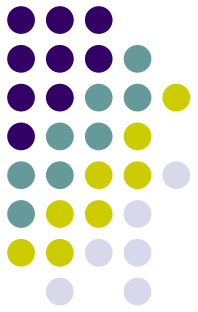
# 相关函数——semctl

该函数用来直接控制信号量信息，它的原型为

```
int semctl(int sem_id, int sem_num, int command, ...);
```

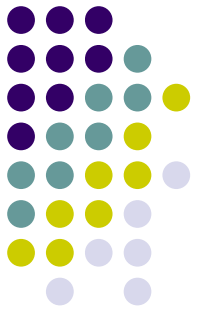
第四个参数，它通常是一个union semum结构，定义如下：

```
union semun{  
    int val;  
    struct semid_ds *buf;  
    unsigned short *array;  
};
```



# 使用举例——例5-12sem1.c

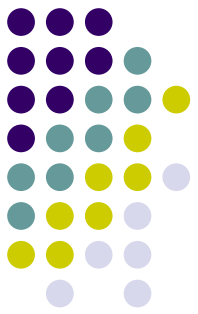
```
int main(int argc, char *argv[])
{ char message = 'X';    int i = 0;
  if(argc > 1)          message = argv[1][0];
  for(i = 0; i < 10; ++i)
  { printf("%c", message);
    fflush(stdout);
    sleep(rand() % 3);
    printf("%c", message);
    fflush(stdout);
    sleep(rand() % 2);
  }
  sleep(10);
  printf("\n%d - finished\n", getpid());
  exit(EXIT_SUCCESS);
}
```



# 使用举例——例5-13sem2.c

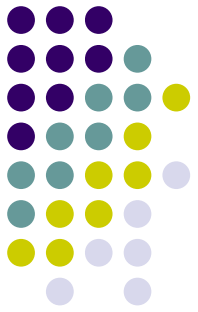
```
static int set_semvalue()
{
    union semun sem_union;
    sem_union.val = 1; //用于初始化信号量
    if(semctl(sem_id, 0, SETVAL, sem_union) == -1)
        return 0;
    return 1;
}

static void del_semvalue()
{
    union semun sem_union;
    if(semctl(sem_id, 0, IPC_RMID, sem_union) == -1)
        fprintf(stderr, "Failed to delete semaphore\n");
}
```



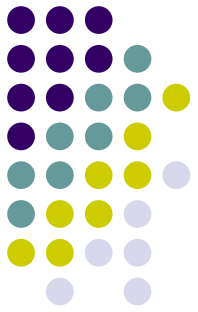
# 使用举例——例5-13sem2.c

```
static int semaphore_p()
{
    //对信号量做减1操作，即等待P（sv）
    struct sembuf sem_b;
    sem_b.sem_num = 0;
    sem_b.sem_op = -1; //P()
    sem_b.sem_flg = SEM_UNDO;
    if(semop(sem_id, &sem_b, 1) == -1)
    {
        fprintf(stderr, "semaphore_p failed\n");
        return 0;
    }
    return 1;
}
```



# 使用举例——例5-13sem2.c

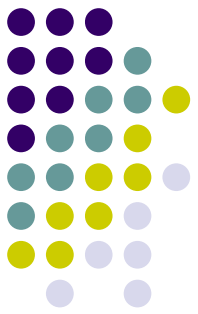
```
static int semaphore_v()  
{  
    //这是一个释放操作，它使信号量变为可用，即发送信号V（sv）  
    struct sembuf sem_b;  
    sem_b.sem_num = 0;  
    sem_b.sem_op = 1;//V()  
    sem_b.sem_flg = SEM_UNDO;  
    if(semop(sem_id, &sem_b, 1) == -1)  
    {  
        fprintf(stderr, "semaphore_v failed\n");  
        return 0;  
    }  
    return 1;  
}
```



# 使用举例——例5-13sem2.c

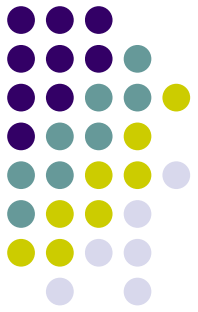
```
union semun
{ int val;    struct semid_ds *buf;    unsigned short *array; };
static int sem_id = 0;
static int set_semvalue();
static void del_semvalue();
static int semaphore_p();
static int semaphore_v();
int main(int argc, char *argv[])
{    char message = 'X';    int i = 0;
    sem_id = semget((key_t)1234, 1, 0666 | IPC_CREAT); //创建信号量
    if(argc > 1)
    { if(!set_semvalue())    //程序第一次被调用，初始化信号量
        { fprintf(stderr, "Failed to initialize semaphore\n");
            exit(EXIT_FAILURE); }
        message = argv[1][0]; //设置要输出到屏幕中的信息，即其参数的第一个字符
        sleep(2);
    }
}
```





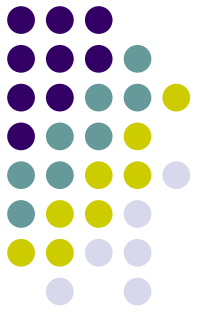
# 使用举例——例5-13sem2.c

```
for(i = 0; i < 10; ++i)
{
    if(!semaphore_p()) //进入临界区 前执行P操作
        exit(EXIT_FAILURE);
    printf("%c", message);      fflush(stdout);      sleep(rand() % 3);
    printf("%c", message); //离开临界区前再一次向屏幕输出数据
    fflush(stdout);
    if(!semaphore_v())
        exit(EXIT_FAILURE); //离开临界区，休眠随机时间后继续循环
    sleep(rand() % 2);
}
sleep(10);
printf("\n%d - finished\n", getpid());
if(argc > 1) { sleep(3); del_semvalue(); }
//如果程序是第一次被调用，则在退出前删除信号量
exit(EXIT_SUCCESS);
}
```



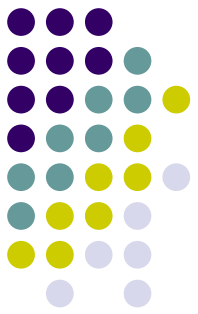
# 掌握的知识点

- IPC通信的处理方式
- 消息队列和共享内存的特点
- 信号量的应用



## Part 4 网络编程

- Socket的基本概念
- 如何实现面向连接的TCP编程
- 如何实现非连接的UDP编程
- 复杂网络程序的实现
- 网络程序的应用



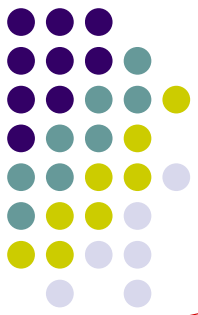
# 4.0 服务器和客户机的信息函数

## 1. 字节序列转换

- 每一台机器内部对变量的字节存储顺序不同，而网络传输的数据是一定要统一顺序的。所以对内部字节表示顺序与网络字节顺序（大端）不同的机器，一定要对数据进行转换

以0x12345678数据为例，两种模式在内存中的存储情况。

| 内存地址   | 小端模式存储内容 | 大端模式存储内容 |
|--------|----------|----------|
| 0x0000 | 0x78     | 0x12     |
| 0x0001 | 0x56     | 0x34     |
| 0x0002 | 0x34     | 0x56     |
| 0x0003 | 0x12     | 0x78     |



## 4.0 服务器和客户机的信息函数

- 头文件: `include <arpa/inet.h>`

短整型

host network → short int

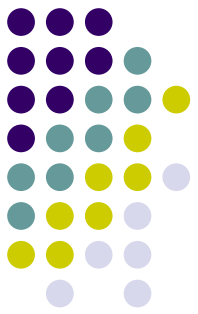
- `unsigned short int htons` (`unsigned short int hostshort`):  
主机字节顺序转换成网络字节顺序, 对无符号短型进行操作  
4bytes

short int S  
long int L

`unsigned long int htonl` (`unsigned long int hostlong`):  
主机字节顺序转换成网络字节顺序, 对无符号长型进行操作  
8bytes

`unsigned short int ntohs` (`unsigned short int netshort`):  
网络字节顺序转换成主机字节顺序, 对无符号短型进行操作  
4bytes

`unsigned long int ntohl` (`unsigned long int netlong`):  
网络字节顺序转换成主机字节顺序, 对无符号长型进行操作  
8bytes



## 4.0 服务器和客户机的信息函数

### 2.地址格式转换

头文件: `#include <sys/types.h>`  
`#include <sys/socket.h>`  
`#include <arpa/inet.h>`

**int inet\_pton**(int family, const char \*strptr, void \*addrptr);

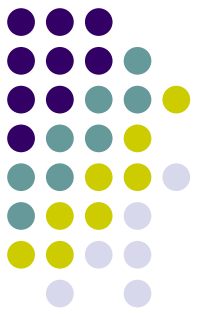
将点分文本的IP地址转换为二进制网络字节序的IP地址

返回: 1成功; -1出错

**const char\* inet\_ntop**(int family, const void \*addrptr, char \*strptr, size\_t len);

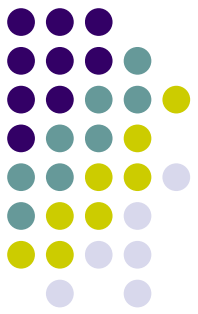
转换网络二进制结构到ASCII类型的地址

返回: 成功, 指向结果的指针; 出错, NULL



## 4.1 Socket 基础

- **socket**是网络编程的一种接口，它是一种特殊的I/O，用**socket**函数建立一个**Socket**连接，此函数返回一个整型的**socket**描述符，随后进行数据传输。
- 一个**IP**地址，一个通讯端口，就能确定一个通讯程序的位置。为此开发人员专门设计了一个套接结构，就是把网络程序中所用到的网络地址和端口信息放在一个结构体中。
- **UDP**是一个无连接协议，**TCP**是个可靠的端对端协议。传输**UDP**数据包时，**LINUX**不知道也不关心它们是否已经安全到达目的地，而传输**TCP**数据包时，则应先建立连接以保证传输的数据被正确接收。

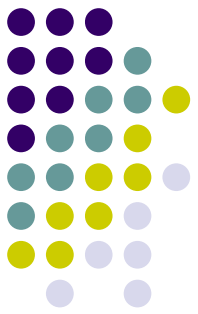


## 4.1 Socket 基础

### 三种类型套接字

- 流套接字(SOCK\_STREAM)
  - 可靠的、面向连接的通信。
  - 使用TCP协议
- 数据报套接字(SOCK\_DGRAM)
  - 无连接服务
  - 使用UDP协议
- 原始套接字(SOCK\_RAW)
  - 允许对底层协议如IP、ICMP直接访问



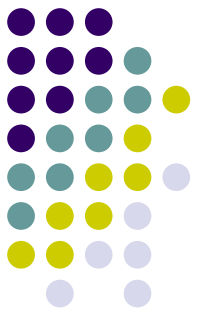


# 1. socket套接字的数据结构

- 两个重要的数据类型：`sockaddr`和`sockaddr_in`，这两个结构类型都是用来保存socket信息的，如IP地址、通信端口等。

**sockaddr——虚拟定义的地址（取决于协议）：**

```
struct sockaddr
{ unsigned short sa_family;
    /*可以为AF_INET，代表TCP/IP地址族*/
    char sa_data[14];
    /*14个字节，包含IP地址和端口号*/
};
```

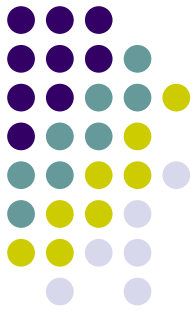


# 1. socket套接字的数据结构

- 两个重要的数据类型：sockaddr和sockaddr\_in，这两个结构类型都是用来保存socket信息的，如IP地址、通信端口等。

sockaddr\_in（AF\_INET中使用的地址定义）：

```
struct sockaddr_in
{
    short sin_family;      /*AF_INET（地址族）*/
    unsigned short sin_port;
    /*端口号(必须要采用网络数据格式)*/
    struct in_addr sin_addr;
    /*网络字节序的IP地址*，只有一个成员s_addr/
    unsigned char sin_zero[8];
    /*与SOCKADDR结构保持同样大小*/ 均无字节。
};
```



## 2. 基于连接的服务

创建一个空的套接字，  
返回文件描述符

将空套接字和本机  
IP地址和端口号绑定，之后socket成为一个三元组

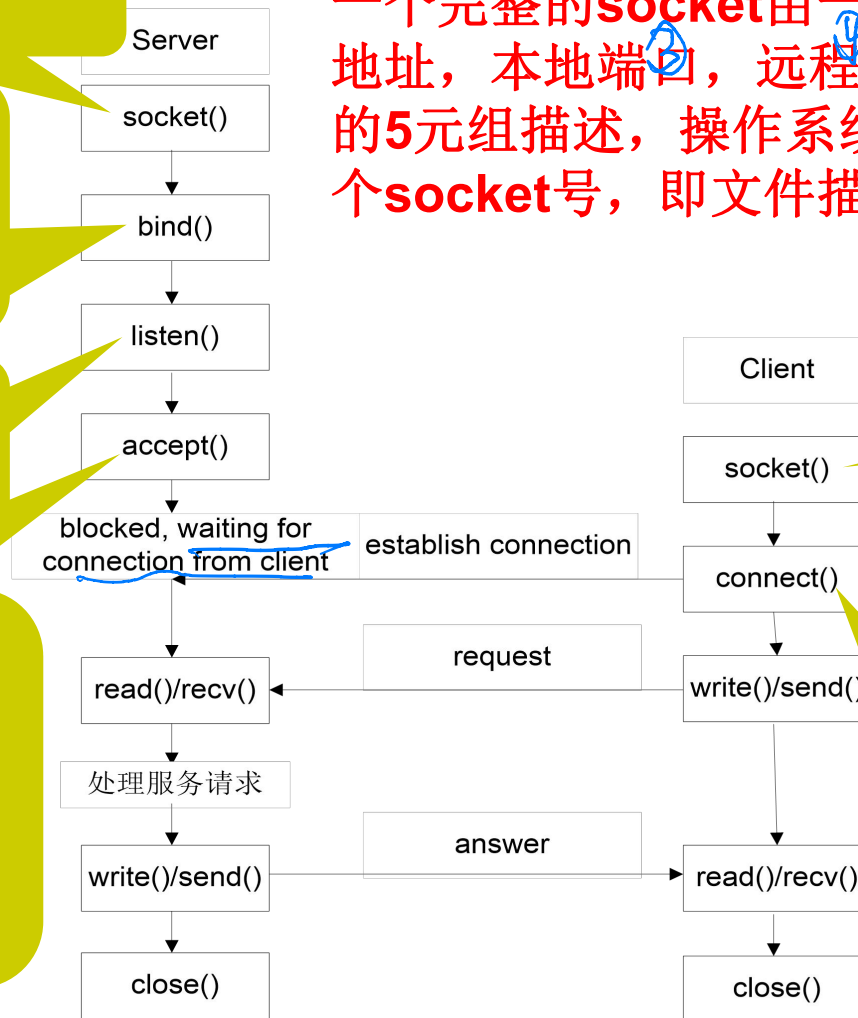
指定一个socket为  
监听套接口

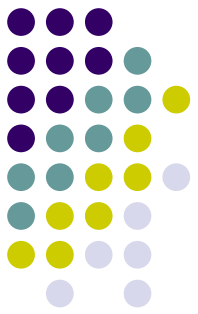
使socket可以接收网络  
连接请求，没有客  
户机连接则阻塞，有  
连接则返回一个包含  
客户机IP和端口号的5  
元组套接字

一个完整的socket由一个{协议，本地地址，本地端口，远程地址，远程端口}的5元组描述，操作系统分配会分配一个socket号，即文件描述符

创建一个空的套接字，  
返回文件描述符

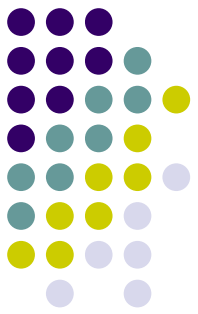
将服务器IP和端口号作为  
参数传入，连接成功  
后，获得和服务端连接  
的一个5元组套接字





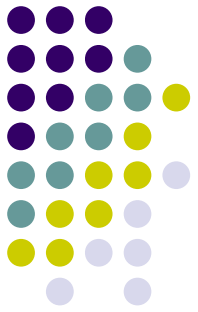
# Server程序的作用

- 程序初始化
- 持续监听一个固定的端口
- 收到Client的连接后建立一个socket连接
- 与Client进行通信和信息处理
  - 接收Client通过socket连接发送来的数据，进行相应处理并返回处理结果
  - 通过socket连接向Client发送信息
- 通信结束后中断与Client的连接



# Client程序的作用

- 程序初始化
- 连接到某个**Server**上，建立**socket**连接
- 与**Server**进行通信和信息处理
  - 接收**Server**通过**socket**连接发送来的数据，进行相应处理
  - 通过**socket**连接向**Server**发送请求信息
- 通信结束后中断与**Server**的连接

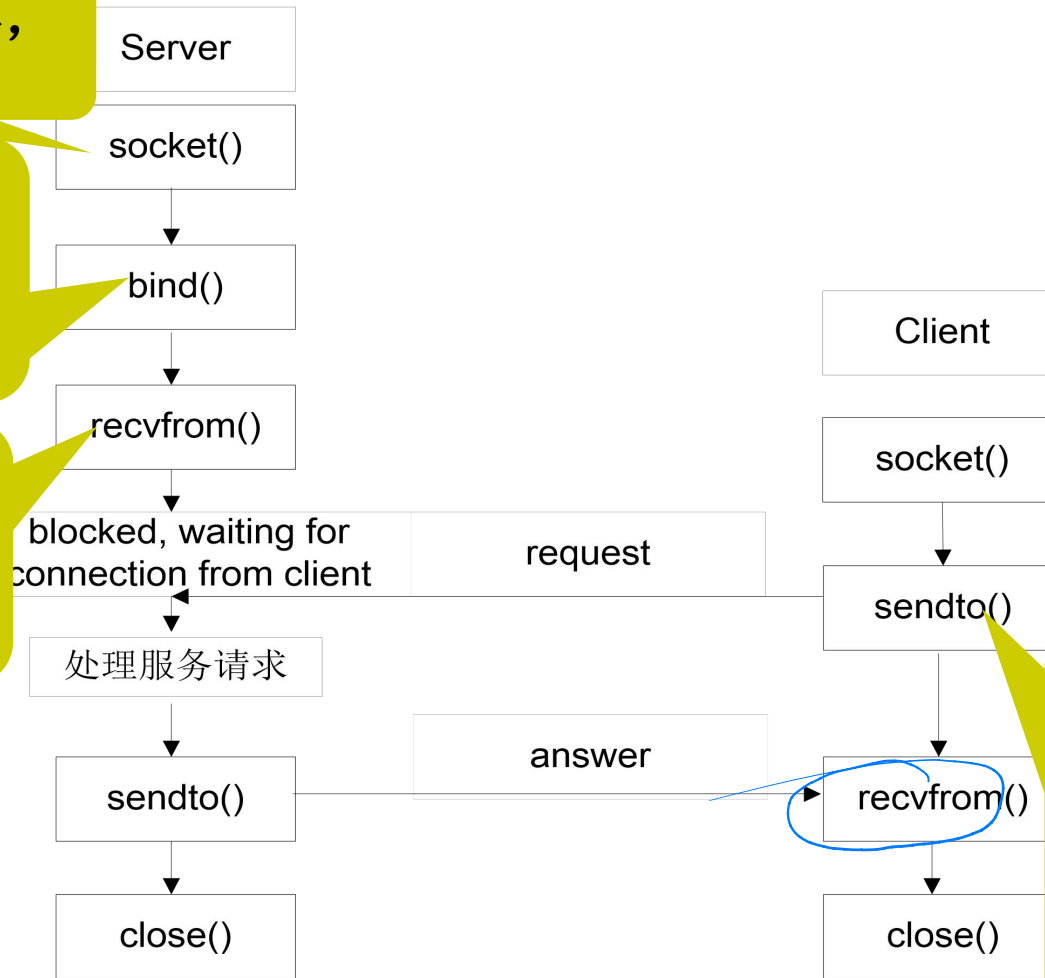


# 3. 无连接的服务

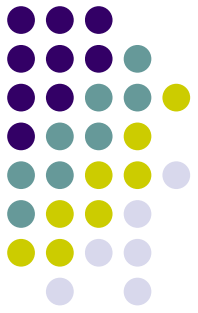
创建一个空的套接字，  
返回文件描述符

将空套接字和本机  
**IP**地址和端口号绑  
定，之后**socket**成  
为一个三元组

从套接字接收发送  
端发来的数据，同  
时获得发送端的**IP**  
地址和端口号

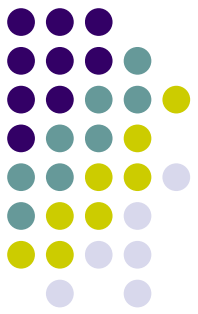


向套接字发送数据，  
同时将接收端的**IP**  
地址和端口号作为  
参数传入，首次发  
送时，**OS**将本机  
**IP**地址和端口号绑  
定到**socket**



# UDP编程的适用范围

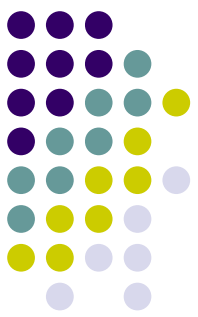
- 部分满足以下几点要求时，应该用UDP
  - 面向数据报
  - 网络数据大多为短消息
  - 拥有大量Client
  - 对数据安全性无特殊要求
  - 网络负担非常重，但对响应速度要求高
- 例子：ICQ、视频点播



# 具体编程时的区别

- `socket()`的参数不同
- UDP Server不需要调用`listen`和`accept`
- UDP收发数据用`sendto/recvfrom`函数
- TCP: 地址信息在`connect/accept`时确定
- UDP: 在`sendto/recvfrom`函数中每次均需指定地址信息
- UDP: `shutdown`函数无效



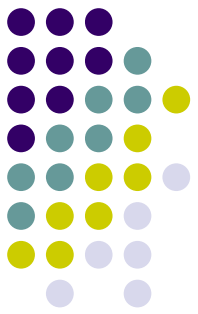


## 4.2 TCP编程

- 基于TCP协议的编程，其最主要的特点是建立完连接后，才进行通信。
- 常用的基于TCP网络编程函数及功能

| 函数名     | 功能                                                                      |
|---------|-------------------------------------------------------------------------|
| bind    | 将 socket 与本机上的一个端口绑定，随后就可以在该端口监听服务请求                                    |
| connect | 面向连接的客户程序使用 connect 函数来配置 socket 并与远端服务器建立一个 TCP 连接                     |
| listen  | listen函数使 socket处于被动的监听模式，并为该 socket建立一个输入数据队列，将到达的服务请求保存在此队列中，直到程序处理它们 |
| accept  | accept()函数让服务器接收客户的连接请求                                                 |
| close   | 停止在该 socket 上的任何数据操作                                                    |
| send    | 数据发送函数                                                                  |
| recv    | 数据接收函数                                                                  |

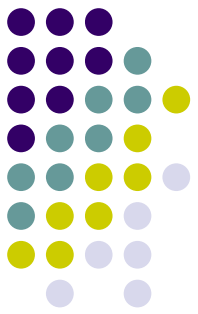
# socket



socket: creates an endpoint for communication and returns a descriptor.

```
int socket(int domain, int type, int protocol);
```

- “domain” parameter
  - Specifies a communication domain, that is, selects a protocol family, such as **AF\_INET**, **AF\_INET6** ...
- “type” parameter
  - Specifies the communication semantics. Three main types are: **SOCK\_STREAM**, **SOCK\_DGRAM**, **SOCK\_RAW**
- “protocol” parameter
  - usually 0 (default).

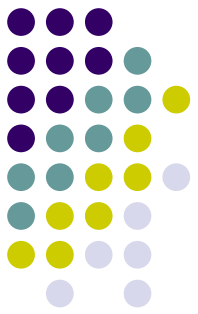


# bind

- bind: binds a name to a socket

```
int bind(int sockfd, struct sockaddr *my_addr,  
        socklen_t addrlen);
```

- struct sockaddr {  
 sa\_family\_t sa\_family;  
 char sa\_data[14];  
}



# listen

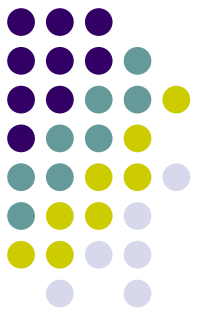
- listen: listen for connections on a socket

- `int listen(int s, int backlog);`

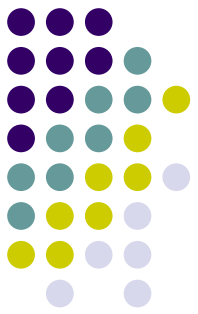
参数说明:

- **s**: `socket()`返回的套接口文件描述符。
- **backlog**: 进入队列中允许的连接个数。  
进入的连接请求在使用系统调用**accept()**应答之前要在进入队列中等待。这个值是队列中最多可以拥有的请求的个数。大多数系统的缺省设置为**20**。

# accept



- **accept()**函数将响应连接请求，建立连接
  - `int accept(int sockfd, struct sockaddr *addr, int *addrlen);`
  - **accept**缺省是阻塞函数，阻塞直到有连接请求
  - **sockfd**: 被动(倾听)的**socket**描述符
  - 如果成功，返回一个新的**socket**描述符(**connected socket descriptor**)来描述该连接。这个连接用来与特定的**Client**交换信息
  - **addr**将在函数调用后被填入连接对方的地址信息，如对方的**IP**、端口等。

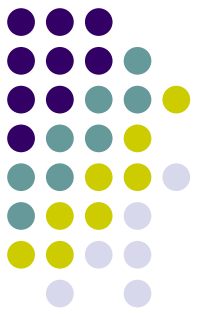


# connect

- connect: initiate a connection on a socket (connect to a server).

```
int connect(int sockfd, struct sockaddr *servaddr, int addrlen);
```

- 主动的socket
- servaddr是事先填写好的结构，Server的IP和端口都在该数据结构中指定。



# send/recv

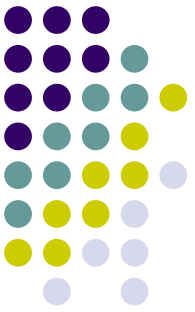
send/recv: connection-oriented

`int send(int s, const void *msg, size_t len, int flag);`

- **s**: 发送数据的套接口文件描述符。
- **msg**: 发送的数据的指针
- **len**: 数据的字节长度
- **flag**: 标志设置为0。

`int recv(int s, void *buf, size_t len, int flag);`

- **s**: 读取的套接口文件描述符。
- **buf**: 保存读入信息的地址。
- **len**: 缓冲区的最大长度。
- **flag**: 设置为0。



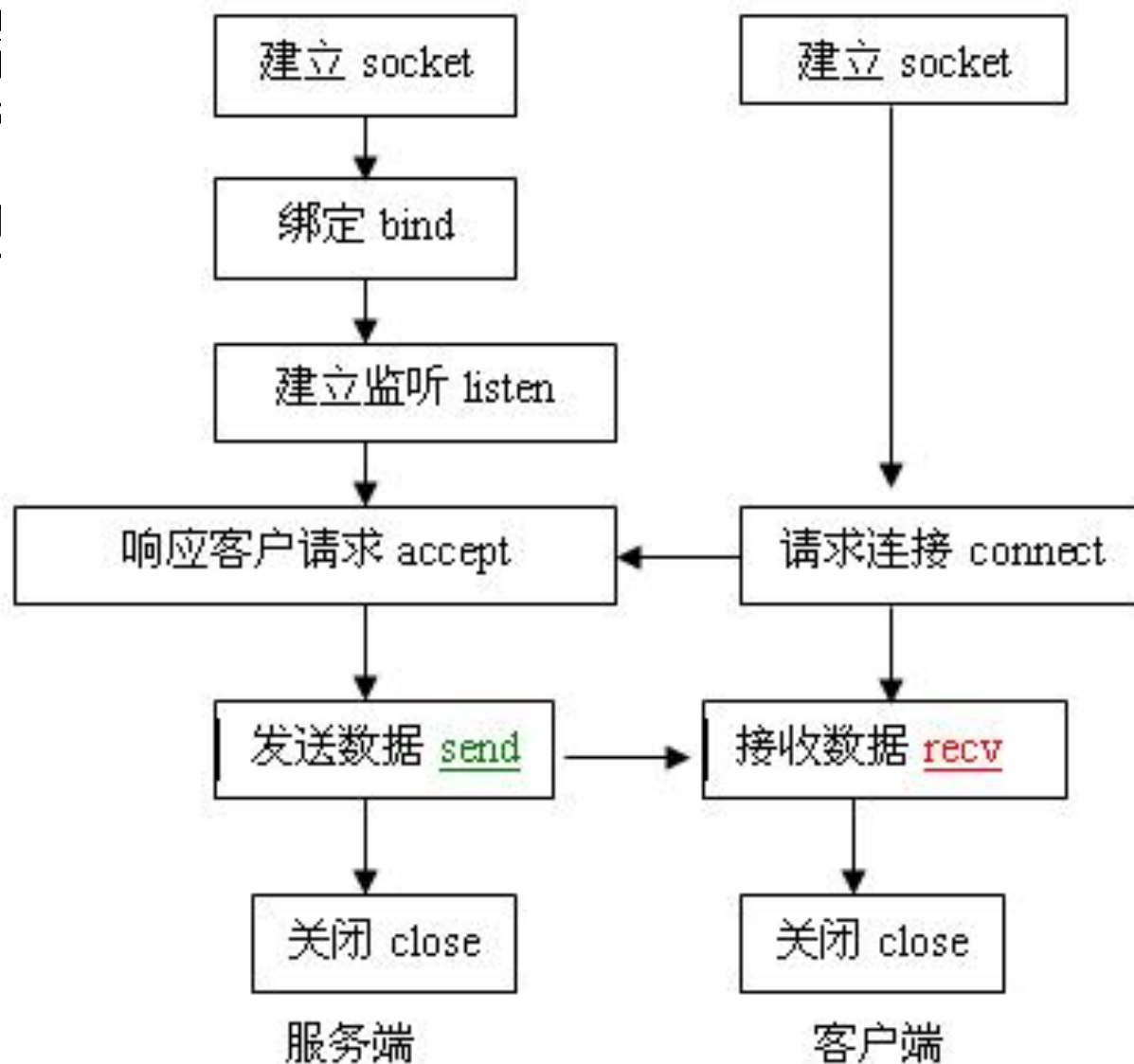
# close & shutdown

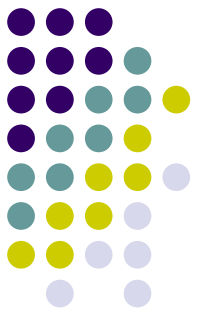
- close
  - `int close(int sockfd);`
- shutdown
  - `int shutdown(int sockfd, int how);`
  - how: SHUT\_RD, SHUT\_WR, SHUT\_RDWR
- shutdown直接对TCP连接进行操作，close只是对套接字描述符操作。



## 4.2 TCP编程

- **例5-14:** 服务器端发送字符串“示客户端的IP地

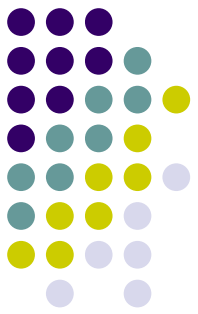




## 4.2 TCP编程

例5-14程序中的主要语句说明:

- 服务端
  - 建立socket: `socket(AF_INET, SOCK_STREAM, 0);`
  - 绑定bind: `bind(sockfd, (struct sockaddr *)&my_addr, sizeof(struct sockaddr));`
  - 建立监听listen: `listen(sockfd, BACKLOG);`
  - 响应客户请求: `accept(sockfd, (struct sockaddr *)&remote_addr, &sin_size);`
  - 发送数据send: `send(client_fd, "连接上了 \n", 26, 0);`
  - 关闭close: `close(client_fd);`



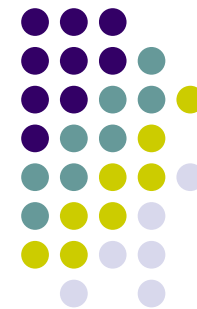
## 4.2 TCP编程

### 客户端:

- ① 建立socket: `socket(AF_INET, SOCK_STREAM, 0);`
- ② 请求连接connect: `connect(sockfd, (struct sockaddr *)&serv_addr, sizeof(struct sockaddr));`
- ③ 接收数据recv: `recv(sockfd, buf, MAXDATASIZE, 0);`
- ④ 关闭close: `close(sockfd);`

## 4.2 TCP编程

```
#include<stdio.h>
#include<stdlib.h>
#include<errno.h>
#include <string.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include <sys/wait.h>
#include <arpa/inet.h>
#define SERVPORT 3333
#define BACKLOG 10
```



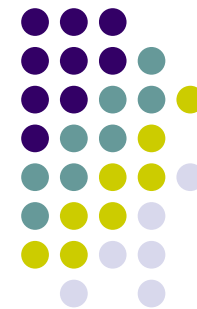
服务端源程序代码:

```
int main()
{
    int sockfd, client_fd;    int sin_size;
    struct sockaddr_in my_addr;    struct sockaddr_in remote_addr;
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    my_addr.sin_family=AF_INET;
    my_addr.sin_port=htons(SERVPORT);
    my_addr.sin_addr.s_addr = INADDR_ANY;
    bzero(&(my_addr.sin_zero), 8);
    if (bind(sockfd, (struct sockaddr *)&my_addr, sizeof(my_addr)) == -1)
    {
        perror("bind 出错!");
        exit(1);
    }
    if (listen(sockfd, BACKLOG) == -1)
    {
        perror("listen 出错!");
        exit(1);
    }
}
```

Handwritten notes in Chinese:

- AF\_INET: 网络地址族
- SOCK\_STREAM: 流式套接字
- INADDR\_ANY: 本机地址
- bzero(&(my\_addr.sin\_zero), 8): 8字节填充为0
- bind: 绑定
- listen: 监听

## 4.2 TCP编程



```
while(1)
```

```
{  
    sin_size = sizeof(struct sockaddr_in);  
    if ((client_fd = accept(sockfd, (struct sockaddr *)&remote_addr,  
        &sin_size)) == -1)  
    {  
        perror("accept error");  
        continue;  
    }  
    printf("收到一个连接来自:  %s\n",  
        inet_ntoa(remote_addr.sin_addr));  
    if (!fork())  
    {  
        if (send(client_fd, "连接上了\n", 26, 0) == -1)  
            error("send 出错! ");  
        close(client_fd);  
        exit(0);  
    }  
    close(client_fd);  
}
```

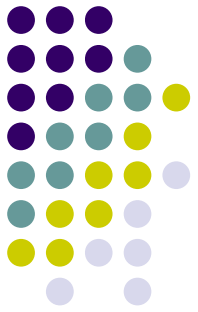
非阻塞 同步阻塞  
→

## 4.2 TCP编程

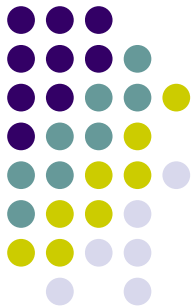
客户端源程序代码：

```
int main(int argc, char *argv[]){
    int sockfd, recvbytes;
    char buf[MAXDATASIZE];
    struct hostent *host;
    struct sockaddr_in serv_addr;
    if (argc < 2) { fprintf(stderr, "Please enter the server's hostname!\n");
        exit(1); }
    if((host=gethostbyname(argv[1]))==NULL)
        { perror("gethostbyname error! "); exit(1); }
    if ((sockfd = socket(AF_INET, SOCK_STREAM, 0)) == -1)
        { perror("socket create error! "); exit(1); }
    serv_addr.sin_family=AF_INET;
    serv_addr.sin_port=htons(SERVPORT);
    serv_addr.sin_addr = *((struct in_addr *)host->h_addr);
    bzero(&(serv_addr.sin_zero),8);
```

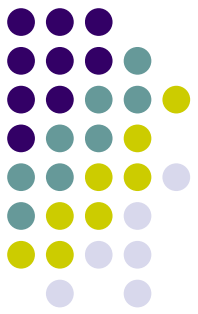
```
#include<stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>
#include <netdb.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#define SERVPORT 3333
#define MAXDATASIZE 100
```



## 4.2 TCP编程



```
if (connect(sockfd, (struct sockaddr *)&serv_addr,
sizeof(struct sockaddr)) == -1) {
    perror("connect error! ");
    exit(1);
}
if ((recvbytes=recv(sockfd, buf, MAXDATASIZE, 0)) ==-1)
{
    perror("connect 出错! ");
    exit(1);
}
buf[recvbytes] = '\0';
printf("收到: %s",buf);
close(sockfd);
}
```



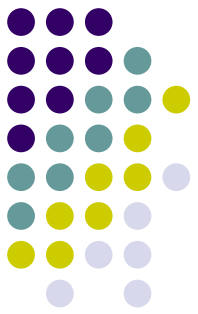
## 4.3 UDP编程

- 基于UDP协议的编程，其最主要的特点是不需要用函数bind把本地IP地址与端口号进行绑定，也能进行通信。
- 常用的基UDP网络编程函数及功能：

| 函数名      | 功能                                   |
|----------|--------------------------------------|
| bind     | 将 socket 与本机上的一个端口绑定，随后就可以在该端口监听服务请求 |
| close    | 停止在该 socket 上的任何数据操作                 |
| sendto   | 数据发送函数                               |
| recvfrom | 数据接收函数                               |



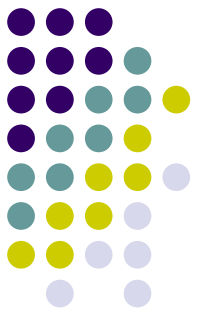
# sendto/recvfrom



- sendto/recvfrom

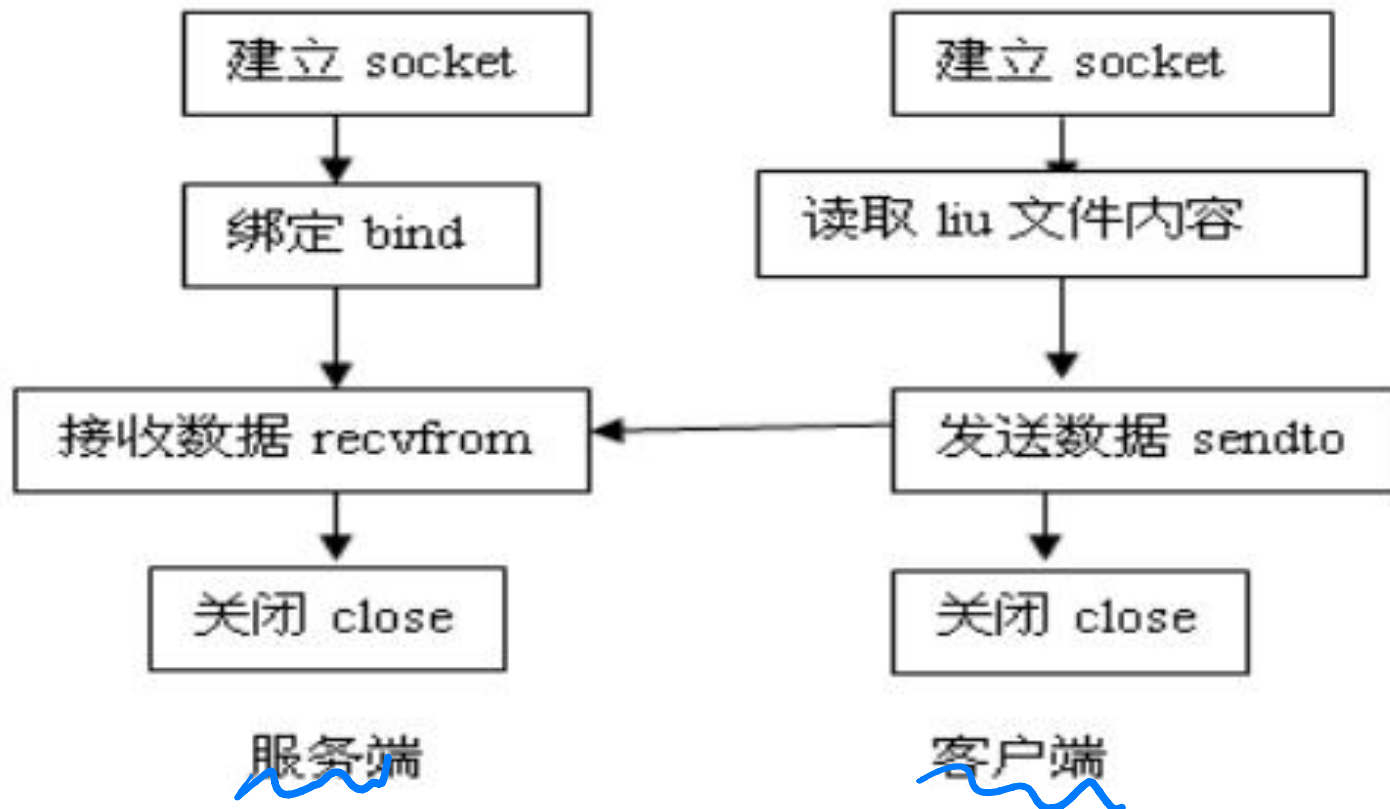
```
int sendto(int s, const void *msg, size_t len, int flags,  
           const struct sockaddr *to, socklen_t tolen);
```

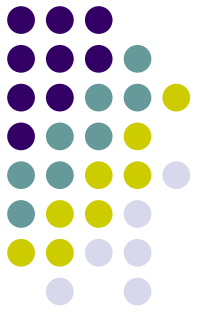
```
int recvfrom(int s, void *buf, size_t len, int flags,  
            struct sockaddr *from, socklen_t *fromlen);
```



## 4.3 UDP编程

- **例5-15：** 服务器端接受客户端发送的字符串。客户端将打开liu文件，读取文件中的 3 个字符串，发送给服务器端，当发送给服务端的字符串为” stop”时，终止数据传送并断开连接。





## 4.3 UDP编程

- 主要语句说明:

### 服务端:

① 建立socket:

```
socket(AF_INET, SOCK_DGRAM, 0)
```

② 绑定bind: `bind(sockfd, (struct sockaddr *)&adr_inet, sizeof(adr_inet));`

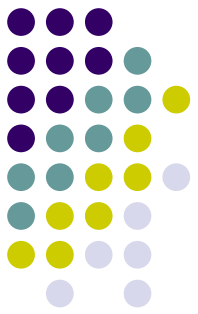
③ 接收数据recvfrom:

```
recvfrom(sockfd, buf, sizeof(buf), 0, (struct  
sockaddr *)&adr_clnt, &len);
```

④ 关闭close: `close(sockfd);`

接收数据

接收数据



## 4.3 UDP编程

- 客户端:

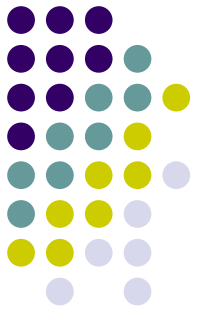
- ① 建立socket: `socket(AF_INET, SOCK_STREAM, 0);`
- ② 读取liu文件: `fopen("liu","r");`
- ③ 发送数据sendto:  
`sendto(sockfd,buf,sizeof(buf),0,(struct sockaddr *)&adr_srvr,sizeof(adr_srvr));` 数据发送
- ④ 关闭close: `close(sockfd);`

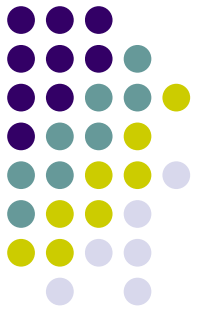
## 4.3 UDP编程

服务端源程序代码:

```
int port=8888;
int main()
{  int sockfd;  int len; int z;
   struct sockaddr_in adr_inet;
   struct sockaddr_in adr_clnt; char buf[256];
   printf("等待客户端....\n");
   adr_inet.sin_family=AF_INET;
   adr_inet.sin_port=htons(port);
   adr_inet.sin_addr.s_addr =htonl(INADDR_ANY);
   bzero(&(adr_inet.sin_zero),8);
   len=sizeof(adr_clnt);
   sockfd=socket(AF_INET,SOCK_DGRAM,0);
   if(sockfd== -1)
   {   perror("socket 出错");           exit(1); }
      z=bind(sockfd,(struct sockaddr *)&adr_inet,sizeof(adr_inet));
```

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<sys/socket.h>
#include<netinet/in.h>
#include<arpa/inet.h>
#include<netdb.h>
#include<errno.h>
#include<sys/types.h>
```





## 4.3 UDP编程

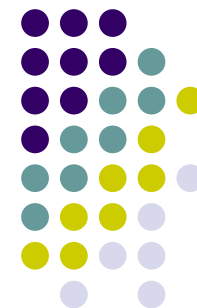
```
if(z==-1)
{
    perror("bind 出错");
    exit(1); }
while(1)
{
    z=recvfrom(sockfd,buf,sizeof(buf),0,(struct sockaddr
    *)&adr_clnt,&len);
    if(z<0)
    {
        perror("recvfrom 出错");
        exit(1);
    }
    buf[z]=0;
    printf("接收:%s",buf);
    if(strncmp(buf,"stop",4)==0)
    {
        printf("结束....\n");
        break; }
    }
    close(sockfd);
    exit(0);
}
```

## 4.3 UDP编程

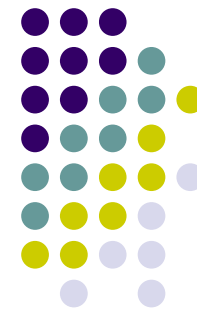
客户端源程序代码：

```
int main()
{
    int sockfd;    int i=0; int z;
    char buf[80],str1[80]; struct sockaddr_in adr_srvr;
    FILE *fp;
    printf("打开文件.....\n");
    fp=fopen("liu","r");
    if(fp==NULL)
    {
        perror("打开文件失败");
        exit(1); }
    printf("连接服务端...\n");
    adr_srvr.sin_family=AF_INET;
    adr_srvr.sin_port=htons(port);
    adr_srvr.sin_addr.s_addr = htonl(INADDR_ANY);
    bzero(&(adr_srvr.sin_zero),8);
    sockfd=socket(AF_INET,SOCK_DGRAM,0);
```

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<sys/socket.h>
#include<netinet/in.h>
#include<arpa/inet.h>
#include<netdb.h>
#include<errno.h>
#include<sys/types.h>
int port=8888;
```

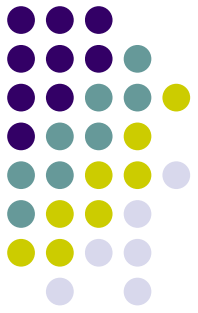


## 4.3 UDP编程



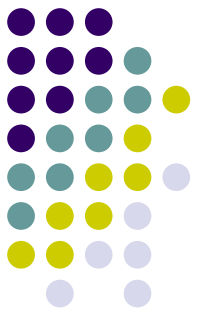
```
if(sockfd==-1)
{
    perror("socket 出错");  exit(1); }
printf("发送文件 ....\n");
for(i=0;i<3;i++)
{
    fgets(str1,80,fp);          printf("%d:%s",i,str1);
    sprintf(buf,"%d:%s",i,str1);
    z=sendto(sockfd,buf,sizeof(buf),0,(struct sockaddr *)&adr_srvr,
    sizeof(adr_srvr));
}
printf("发送.....\n");  sprintf(buf,"stop\n");
z=sendto(sockfd,buf,sizeof(buf),0,(struct sockaddr *)&adr_srvr,
sizeof(adr_srvr));
if(z<0)
{
    perror("sendto 出错");          exit(1); }
fclose(fp);  close(sockfd);  exit(0);
}
```





## 4.4 网络高级编程

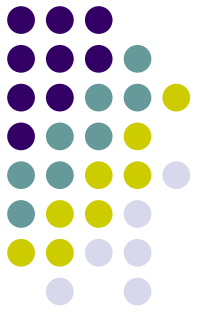
- I/O Models
  - Block mode 阻塞模式
  - Non-block mode 非阻塞模式



# 阻塞方式

- 阻塞方式:在数据通信中，当服务器运行函数 **accept()** 时，~~假设没有客户机连接请求到来，那么服务器就一直会停止在~~**accept**（）语句上，等待客户机连接请求到来，出现这样的情况就称为阻塞。





# 非阻塞方式

- 阻塞与非阻塞方式的比较

- errno - EWOULDBLOCK

- 非阻塞的实现

```
int flags;
```

获得文件标志、

```
if ( (flags=fcntl(sock_fd, F_GETFL, 0)) < 0)
```

fcntl

或者  
← err\_sys();

套接文件

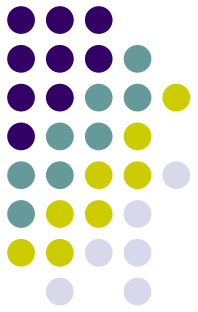
```
flags |= O_NONBLOCK;
```

```
if ( fcntl(sock_fd, F_SETFL, flags) < 0)
```

```
err_sys();
```

重新设置一下标志

非阻塞、



# 掌握的知识点

- TCP客户端和服务端的编程模型
- UDP客户端和服务端的编程模型
- 服务器的多进程并发处理方式